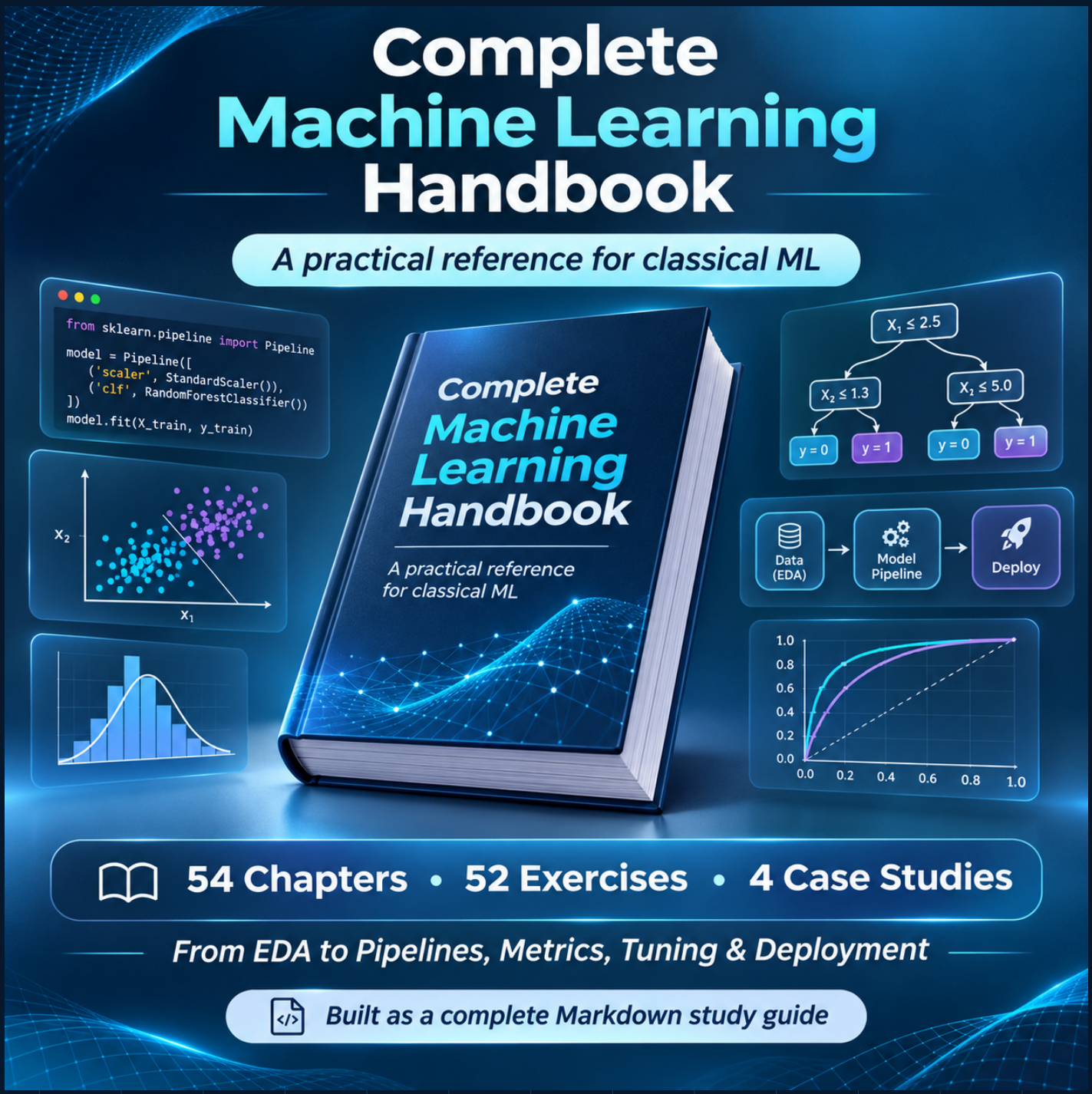
CLASSICAL MACHINE LEARNING



A PRACTICAL STUDY & REFERENCE COLLECTION

Theory / Mathematics / Python / Exercises / Soutenance

Contents

THE COMPLETE CLASSICAL MACHINE LEARNING HANDBOOK **3**

Part I — The Foundations **3**
1. What is machine learning? . . . 3
2. Essential mathematics: everything you use in ML . . . 4
3. Understanding your dataset . . . 6
4. The complete Machine Learning project lifecycle . . . 7
5. Underfitting, overfitting, bias–variance and generalization . . . 8

Part II — Data Understanding & Preparation **9**
6. NumPy and Pandas essentials for ML . . . 9
7. EDA — Exploratory Data Analysis . . . 9
8. Missing values, duplicates, inconsistent observations and outliers . . . 10
9. Feature engineering, selection and extraction . . . 11
10. Encoding, scaling, transformations and imbalance . . . 11
11. Splitting, stratification, cross-validation and leakage . . . 13

Part III — Regression Models **14**
A common regression experiment: reusable pattern . . . 14
12. Simple Linear Regression . . . 14
13. Multiple Linear Regression . . . 15
14. Polynomial Regression . . . 15
15. Ridge, Lasso and Elastic Net . . . 15
16. K-Nearest Neighbors Regression . . . 16
17. Decision Tree Regression . . . 16
18. Random Forest and Extra Trees Regression . . . 17
19. Support Vector Regression (SVR) . . . 17
20. Boosted-tree regression: Gradient Boosting, AdaBoost, XGBoost, LightGBM, CatBoost 18
21. Regression metrics and diagnostics . . . 19

Part IV — Classification Models **20**
How to compare classifiers fairly . . . 20
22. Logistic Regression . . . 21
23. K-Nearest Neighbors (KNN) classification . . . 21
24. Naive Bayes (Gaussian, Multinomial, Bernoulli) . . . 22
25. Decision Tree Classifier . . . 22
26. Random Forest and Extra Trees Classifier . . . 22
27. Support Vector Machines (SVM) . . . 23
28. Linear Discriminant Analysis (LDA) and Quadratic Discriminant Analysis (QDA) . . . 23
29. Boosting classifiers: AdaBoost, Gradient Boosting, XGBoost, LightGBM and CatBoost 23
30. SGD-based linear classifiers . . . 24
31. Binary, multiclass and multilabel classification . . . 24
32. Classification metrics and threshold selection . . . 25

Part V — Unsupervised Learning **27**
33. Clustering fundamentals and algorithm choice . . . 27
34. K-Means and K-Means++ . . . 28
35. DBSCAN and hierarchical clustering . . . 28
36. Gaussian Mixture Models (GMM) . . . 29
37. Evaluating clustering . . . 29
38. Dimensionality reduction: PCA, t-SNE, UMAP and SVD . . . 30
39. Anomaly detection: Isolation Forest, LOF, One-Class SVM . . . 31

Part VI — Professional ML Workflows **32**
40. Scikit-learn Pipeline, ColumnTransformer and reusable preprocessing . . . 32
41. Cross-validation, GridSearchCV, RandomizedSearchCV and tuning . . . 33
42. Model comparison and baselines . . . 34
43. Interpreting models: coefficients, importance, permutation, SHAP . . . 35
44. Save and load a complete model with Joblib . . . 35
45. MLflow experiment tracking . . . 35

46. Model deployment: FastAPI and Streamlit . . . 36
47. Monitoring, drift, reproducibility and responsible ML . . . 37

Part VII — Soutenance Croisée Preparation **37**
48. Model comparison — which algorithm, and why? . . . 37
49. Oral technical questions — short, precise answers . . . 39
50. Defending your ML project — oral presentation framework . . . 41
51. Code review: spot the ML mistakes . . . 42

Chapter 52 — Exercises, challenges and mini-projects **43**
Level A — Foundations, probability and statistics . . . 44
Level B — EDA, preprocessing and validation . . . 44
Level C — Regression and mathematical reasoning . . . 45
Level D — Classification and metrics . . . 45
Level E — Clustering, PCA and anomalies . . . 46
Level F — Professional ML and debugging . . . 46
Capstone mini-projects (graded by reasoning, not headline score) . . . 47
Self-grading rubric (100 pts) . . . 47

Chapter 53 — Worked Solutions and Corrections **47**
Level A — Foundations . . . 48
Level B — Data preparation . . . 48
Level C — Regression . . . 49
Level D — Classification . . . 50
Level E — Unsupervised . . . 51
Level F — Professional workflow . . . 51
Capstone review guide . . . 52

Chapter 54 — Final Revision Cheat Sheets **52**
The ML lifecycle in 12 verbs . . . 52
Task-to-algorithm lookup . . . 52
Key equations — regression . . . 53
Key equations — classification . . . 53
Key equations — unsupervised . . . 54
Model parameters that examiners ask about . . . 54
Quick, safe Python recipes . . . 54
Error-pattern quick diagnosis . . . 55
10 sentences worth memorizing . . . 55
90-second exam readiness checklist . . . 56

Appendix — Other Classical ML Topics Worth Recognizing **56**
A. Ensemble voting, bagging and stacking . . . 56
B. Robust, count, quantile and generalized linear regression . . . 56
C. Time series forecasting without deep learning . . . 57
D. Semi-supervised learning and active learning . . . 57
E. Association rules and frequent patterns . . . 57
F. Recommendation and ranking . . . 57
G. Survival analysis and censoring . . . 57
H. Multioutput, sample weights, cost and distribution shift . . . 58
I. Predictive uncertainty and conformal methods . . . 58

Official References and API Verification **58**
Core Python and scikit-learn . . . 58
Gradient boosting, interpretation and visualization . . . 58
Tracking and deployment . . . 58
Other classical methods . . . 59

Case Study 1 — Price Prediction (Regression) **59**
1. Business framing and target . . . 59
2. Data dictionary example . . . 59
3. EDA plan . . . 60
4. Validation design . . . 60

5. Modeling and scorecard 60
6. Oral defense questions 60

Case Study 2 — Customer Churn Prediction (Classification) 60
1. Important business questions 60
2. Dataset 60
3. EDA and leakage audit 61
4. Split and pipeline 61
5. Compare candidates 61
6. Threshold calculation example 61
7. Explain the result 61

Case Study 3 — Customer Segmentation (Unsupervised) 62
1. Define similarity 62
2. EDA and preparation 62
3. Compare K-Means, DBSCAN and GMM 62
4. Validate usefulness 62
5. Using clusters as supervised features 62
6. Oral questions 63

Case Study 4 — Anomaly Detection + Operational Monitoring 63
1. Define abnormality and decision cost 63
2. Data pipeline 63
3. Model comparison 63
4. Deployment interface 63
5. Monitoring plan 64

THE COMPLETE CLASSICAL MACHINE LEARNING HANDBOOK

The entire reference book in one file · Prepared for *soutenance croisée* · No deep learning · November 2026

This standalone PDF edition contains the chapters, exercises, solutions, case studies and reference appendix. Its contents and bookmarks are clickable.

Part I — The Foundations

Chapters 1–5 · Mental model, mathematics, datasets, ML lifecycle, generalization.

1. What is machine learning?

Intuition. Traditional programming specifies rules (`if condition: output`). Machine learning estimates rules from examples. Given observations X, outcomes y, and a hypothesis family f_θ, a learning algorithm searches for parameters θ that generalize to unseen cases. Training accuracy is *not* the ultimate objective; trustworthy performance on future data is.

Vocabulary. An *observation/sample* is one row; a *feature* is an input column; a *label/target* is what we want to predict; a *parameter* is learned from data (e.g., regression coefficient); a *hyperparameter* is chosen before/during model selection (e.g., tree depth). `fit()` learns, `predict()` generates labels/values, `predict_proba()` generates estimated class probabilities where supported. Predictions are not automatically causal conclusions.

Paradigm	Learning signal	Typical objective	Example	Common algorithms
Supervised	Input + target	Predict an outcome	Price/churn	Linear/logistic regression, trees, SVM, boosted ensembles
Unsupervised	Inputs only	Discover patterns	Segments	K-Means, DBSCAN, PCA, GMM

Paradigm	Learning signal	Typical objective	Example	Common algorithms
Semi-supervised	Few labels + many unlabeled rows	Exploit unlabeled structure	Label-scarce classification	Self-training, label propagation
Self-supervised	Pseudo-target derived from data	Learn representations	Predict hidden or future input parts	Broad family, not necessarily deep learning
Reinforcement	Rewards after actions	Learn behavior in an environment	Sequential control	Tabular Q-learning, bandits, policy methods
Online/incremental	Streaming data	Update progressively	Large/continuously arriving data	SGD, `partial_fit` estimators
Batch/offline	Fixed training set	Train then deploy	Most tabular projects	Most scikit-learn estimators

Task type is different from paradigm: Regression predicts numerical values; classification predicts categories; ranking orders candidates; clustering discovers groups; anomaly detection identifies atypical patterns; dimensionality reduction compresses features. Time-series forecasting may be supervised but requires *time-aware* validation. Recommendation can combine several paradigms.

Learning objective: Empirical risk minimization chooses parameters to reduce average loss:

$$\hat{\theta} = \arg\min_{\theta} \left[\frac{1}{n} \sum_{i=1}^{n} L(f_\theta(x_i), y_i) + \lambda \Omega(\theta) \right].$$

L penalizes wrong predictions; Ω regularizes complexity; λ controls that penalty (definitions vary across estimators). Minimizing training loss without complexity control, representative data, or independent validation can yield a poor model.

Example: Predict house value: continuous target → regression. Predict departure of a customer in 30 days → binary classification. Group users by behavior without predefined groups → clustering. Compress correlated measurements for plotting → PCA.

Soutenance prompts: (i) Why is clustering not classification? (ii) What makes a task supervised? (iii) Why does high training accuracy not prove learning useful patterns? (iv) Can the same dataset support both classification and regression?

2. Essential mathematics: everything you use in ML

2.1 Linear algebra

A **scalar** is one number; **vector** $x \in \mathbb{R}^p$ contains p features; **matrix** $X \in \mathbb{R}^{n \times p}$ contains n rows, p columns. Matrix multiplication follows dimension matching: $(n \times p)(p \times 1) \to (n \times 1)$. A transpose swaps axes: $X^\top \in \mathbb{R}^{p \times n}$.

- Dot product: $w^\top x = \sum_j w_j x_j$; linear prediction: $\hat{y} = w^\top x + b$.
- Euclidean distance: $d_2(x, z) = \sqrt{\sum_j (x_j - z_j)^2}$; Manhattan distance: $d_1(x, z) = \sum_j |x_j - z_j|$.
- Norms: $\|w\|_1 = \sum_j |w_j|$ (Lasso), $\|w\|_2^2 = \sum_j w_j^2$ (Ridge).
- Eigenvectors/eigenvalues describe directions that a square transformation stretches; PCA uses eigenvectors of the covariance matrix (or SVD of centered data).
- Singular-value decomposition: $X = U \Sigma V^\top$; useful for dimensionality reduction, least squares and numerical stability.

Worked example: $x = (2, 3)$, $w = (4, -1)$, $b = 5$, so $w^\top x + b = 4 \cdot 2 - 3 + 5 = 10$. Euclidean distance from $(2, 3)$ to $(5, 7)$ equals $\sqrt{3^2 + 4^2} = 5$.

```
import numpy as np
X = np.array([[1., 2.], [3., 4.], [5., 6.]])
w = np.array([0.5, 2.0])
y_hat = X @ w + 1                # [5.5, 10.5, 15.5]
column_means = X.mean(axis=0)     # [3., 4.]
centered = X - column_means       # broadcasting
print(X.shape, y_hat.shape)
```

2.2 Statistics and distributions

For $x_1, \ldots, x_n$: mean $\bar{x} = \frac{1}{n}\sum_i x_i$; median is middle ordered value; mode is most frequent; range is max−min. Sample variance $s^2 = \frac{1}{n-1}\sum_i (x_i - \bar{x})^2$; sample standard deviation $s = \sqrt{s^2}$. Standard deviation has the *same units* as the original feature; variance uses squared units. Population variance uses divisor n. Standard deviation measures dispersion, **not** whether the average is accurate.

Quantiles split a distribution; Q_1 is 25th percentile, Q_3 is 75th, IQR $= Q_3 - Q_1$. The common 1.5×IQR outlier rule is a *flag for inspection*, not a universal deletion instruction. **Skewness** indicates asymmetric tails; **kurtosis** indicates tail heaviness relative to a reference convention. Histograms reveal distribution shapes that mean/standard deviation can conceal.

Covariance: $\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)]$. **Pearson correlation:** $r = \text{Cov}(X, Y)/(\sigma_X \sigma_Y)$ in $[-1, 1]$ measures *linear association*; Spearman correlation measures monotonic association based on ranks. Correlation does not establish causation, and $r \approx 0$ does not exclude nonlinear relationships.

Common distributions: Bernoulli (0/1), binomial (number of successes), normal/Gaussian (continuous symmetric), Poisson (counts), exponential (waiting time), uniform (constant density on interval). Real datasets need not be normally distributed for many ML models; some inferential assumptions concern **errors/residuals**, not raw input features.

2.3 Probability essentials

- $P(A \mid B) = P(A \cap B)/P(B)$ (conditional probability, $P(B) > 0$).
- Bayes: $P(A \mid B) = P(B \mid A)P(A)/P(B)$.
- Independence: $P(A \cap B) = P(A)P(B)$; conditional independence is a distinct assumption.
- Expected value $E[X] = \sum_x xP(X = x)$ for discrete variables; variance $\text{Var}(X) = E[(X - E[X])^2]$.
- **Base-rate fallacy:** even a good detector produces many false alarms when true events are extremely rare. Always examine class prevalence and positive predictive value.

Worked Bayes example: A condition exists in 1% of cases, test sensitivity is 90%, specificity 95%. $P(+) = .90(.01) + .05(.99) = .0585$; $P(\text{condition} \mid +) = .009/.0585 \approx 15.4\%$. Interpret the result as a probability conditional on these hypothetical assumptions, not a medical conclusion.

2.4 Derivatives, gradients and optimization

Derivative is a rate of change: $d(x^2)/dx = 2x$. The **gradient** of a loss with respect to many coefficients is the vector of partial derivatives. Gradient descent repeatedly updates weights against the gradient:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta J(\theta_t),$$

where η is the learning rate. Too large may diverge; too small may converge slowly. **Batch** uses all rows for each update; **stochastic** uses one; **mini-batch** uses subsets. Many classical

algorithms are *not* optimized with generic gradient descent: trees search for splits; KNN stores examples; random forests bag trees; linear least squares can use algebra/numerical solvers.

MSE gradient for simple linear regression: for $\hat{y}_i = wx_i + b$, $J = \frac{1}{n}\sum_i(\hat{y}_i - y_i)^2$:

$$\frac{\partial J}{\partial w} = \frac{2}{n}\sum_i(\hat{y}_i - y_i)x_i, \qquad \frac{\partial J}{\partial b} = \frac{2}{n}\sum_i(\hat{y}_i - y_i).$$

```
x = np.array([1., 2., 3., 4.])
y = np.array([2., 4., 6., 8.])
w, b, lr = 0., 0., 0.05
for _ in range(2000):
    err = w * x + b - y
    w -= lr * 2 * np.mean(err * x)
    b -= lr * 2 * np.mean(err)
print(round(w, 2), round(b, 2))  # approximately 2, 0
```

Complexity intuition: $O(n)$ scales linearly, $O(n^2)$ quadratically, $O(np)$ with samples and features. Distance search, kernel matrices and exhaustive hyperparameter search become costly as data grows. Memory matters as much as CPU.

2.5 Statistical inference vs machine learning

- **Confidence interval:** uncertainty for a parameter under a statistical model; not a 95% probability that a fixed parameter changes each experiment.
- **Prediction interval:** plausible range for a new observation; includes irreducible variability and is usually wider than a mean-response confidence interval.
- **p-value:** a probability of data at least as extreme under the null, not $P(H_0 \mid data)$.
- **Bootstrap:** resample rows with replacement to estimate a distribution of a statistic, respecting grouping/time structure.
- **Random seed:** controls random draws/reproducibility, does not guarantee identical hardware/library behavior or scientific validity.

3. Understanding your dataset

Shapes and semantics: $X \in \mathbb{R}^{n \times p}$ is conceptual; real tables include strings, booleans, dates and missingness. y may be numeric, class labels, a multioutput matrix, or absent. `df.shape`, `df.dtypes`, `df.info()`, `df.describe(include='all')`, `df.isna().mean()` give first diagnostics.

Type	Examples	Appropriate handling	Trap
Numerical continuous	Age, price, temperature	Impute, possibly scale/log transform	Using identifier as quantity
Discrete count	Purchases, tickets	Count-aware features, review skew	Treating count as category blindly
Nominal categorical	City, contract type	One-hot, supported native category handling	Fake ordinal meaning
Ordinal	Low/medium/high	Order-aware encoding when order is defensible	Assuming equal distances
Binary	Yes/no	0/1 encode	Misinterpreting target direction
Datetime	Event timestamp	Cyclical/time-window features	Future information leakage
Text	Review, description	Bag-of-words/TF-IDF (classical)	Using raw strings as numeric
Identifier	Customer ID	Usually remove from predictors	Memorizing identity
Grouped/repeated	Measurements per person	Group-aware split	Same entity in train and test

Feature vs target leakage: If predicting whether a customer will churn *tomorrow*, a `cancellation_date` recorded *after* that outcome is unavailable at prediction time. A superb AUC from such a feature is suspicious. Define the **prediction moment** and only allow data known then.

Sampling: Train/test may be nonrepresentative due to selection bias, seasonality, concept drift, data acquisition errors or repeated entities. Study class proportions, rare groups, missingness by group, geographic/time coverage and target quality. Never discard unusual rows without understanding whether they are errors or the most important cases.

Useful audit:

```
import pandas as pd
# df = pd.read_csv('data.csv')
# print(df.shape, df.head(), df.info())
# print(df.isna().mean().sort_values(ascending=False))
# print(df.nunique(dropna=False))
# print(df.duplicated().sum())
# print(df['target'].value_counts(normalize=True))  # classification
```

4. The complete Machine Learning project lifecycle

1. **Business objective** — What action follows a prediction? What errors cost the most? Define success in business terms.
2. **Prediction specification** — Unit of prediction, target, prediction horizon, information available at prediction time, and exclusions.
3. **Data provenance** — Where do rows originate? Consent, permission, data dictionary, data quality and sampling bias.
4. **Split design before fitting** — Random/stratified, group-aware or temporal split. Reserve a final test set.
5. **Exploration on training data** — Understand distributions, missingness, imbalance, label problems and suspicious fields.
6. **Preprocessing** — Train-only imputation/encoding/scaling/feature selection placed inside a pipeline.
7. **Baseline** — Dummy model or simple linear/logistic model; demonstrate improvement over naïve predictions.
8. **Candidates + cross-validation** — Compare a small, justified set of methods with an appropriate scoring metric.
9. **Tune and diagnose** — Hyperparameters, learning curves, group-specific errors and threshold tradeoffs.
10. **Lock model and assess once** — Refit the chosen workflow on development data; measure final test performance and uncertainty.
11. **Communicate** — Explain limitations, model card, experiment setup, data limitations and error costs.
12. **Deploy and monitor** — Validate schema, save full pipeline, watch drift and outcomes, retrain when justified; safeguard privacy.

Important ordering: You may inspect raw dataset dimensions, schema, and target availability before splitting; **do not compute data-dependent preprocessing statistics on the entire dataset**. If tuning the *choice* of a transformation by looking at the test set, you have also leaked information. Treat the test set as sealed.

Pipeline mental model: `raw training rows → train-fold preprocessing.fit_transform → estimator.fit → validation-fold preprocessing.transform → estimator.predict → metrics`. Every CV fold must learn preprocessing from that fold's training portion.

Minimal, honest evaluation:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_validate
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
X, y = load_breast_cancer(return_X_y=True)
X_dev, X_test, y_dev, y_test = train_test_split(
    X, y, test_size=.20, stratify=y, random_state=42)
pipe = make_pipeline(SimpleImputer(), StandardScaler(), LogisticRegression(max_iter=2000))
cv = cross_validate(pipe, X_dev, y_dev, cv=5, scoring=['accuracy', 'f1'])
```

```
print('CV accuracy:', cv['test_accuracy'].mean())
pipe.fit(X_dev, y_dev)
print(classification_report(y_test, pipe.predict(X_test)))
```

The example uses a built-in educational dataset; do not interpret its predictive results as a clinical tool.

5. Underfitting, overfitting, bias–variance and generalization

Underfitting means a model is too constrained or inadequately trained to capture the relevant pattern: poor training *and* validation scores. **Overfitting** means a model fits training-specific noise/idiosyncrasy and loses performance on new samples: strong training score, much weaker validation score. Neither is diagnosed from one number alone.

Bias–variance decomposition (squared error, idealized assumptions):

$$\mathbb{E}[(Y - \hat{f}(X))^2] = \mathrm{Bias}[\hat{f}(X)]^2 + \mathrm{Var}[\hat{f}(X)] + \sigma^2.$$

- High bias: too rigid model; low variance may still yield systematic mistakes.
- High variance: predictions change considerably with training sample; often a complex/unregularized model.
- σ^2: irreducible noise under the modeling assumptions.

Symptom	Possible causes	What to try
Train and validation both poor	Underfitting, poor features, noisy labels	Better features, more suitable model, less regularization
Train excellent; validation poor	Overfitting, leakage, nonrepresentative split	Simpler model, regularization, more data, correct split, audit leakage
CV strong; deployment weak	Drift, data quality, label delay, optimistic selection	Monitor, recalibrate, re-evaluate with representative temporal data
Metrics differ wildly across folds	Small data, rare classes, group/time differences	Appropriate CV scheme, more samples, inspect per-fold distributions

Learning curve: Training score and CV score against training-set size. **Validation curve:** Training/CV scores against one hyperparameter. Increasing data may help variance-limited models; it does not automatically fix wrong labels or structurally wrong features.

Regularization: Ridge penalizes coefficient magnitude (L_2), Lasso encourages sparsity (L_1); pruning/maximum tree depth limit trees; `C` in logistic/SVM is *inverse* regularization strength; `k` in KNN and `min_samples_leaf` in trees control smoothness. More complexity is not always better.

Leakage types to identify: train+test scaling, imputation before CV, duplicate or same-patient rows across partitions, future feature/target proxy, oversampling before CV, target encoding on same folds, choosing features using all labels, fitting PCA on test data, repeated leaderboard/test-set tuning. Group or time splits often matter more than algorithm sophistication.

Soutenance Q&A:

- *What is generalization?* Performance on new examples from the intended deployment distribution, not memorization of development rows.
- *Why not just train a deeper tree?* A deeper tree can reduce training error while increasing variance and hurting validation performance.
- *Why is `random_state=42` not a guarantee?* It controls particular pseudo-random operations, but not split appropriateness, all nondeterministic libraries, or data drift.
- *What is data leakage?* Information unavailable in real prediction or held-out evaluation influences model creation, inflating the score.

Checkpoint exercise: Imagine 99% training accuracy but 60% test accuracy. State three possible explanations and a concrete diagnostic for each. See Chapter 53.

Part II — Data Understanding & Preparation

Chapters 6–11 · Learn to understand the data before trusting a model.

6. NumPy and Pandas essentials for ML

NumPy — arrays, axes, vectorized operations

An `ndarray` is a homogeneous n-dimensional array. Essential properties: `shape`, `ndim`, `dtype`, `size`. Axis `0` travels down rows (reduces rows for per-column statistics); axis 1 travels across columns (reduces columns for per-row statistics). Vectorization operates on arrays without Python loops. Broadcasting aligns trailing dimensions; use with care.

```
import numpy as np
A = np.array([[2., 5., 8.], [4., 7., 10.], [6., 9., 12.]])
print(A.shape, A.ndim, A.dtype)
print(A.mean(axis=0))          # column means [4, 7, 10]
print(np.median(A, axis=0))    # column medians [4, 7, 10]
print(A.min(axis=0), A.max(axis=0))
print(A.std(axis=0, ddof=1))   # sample std, 2 for each column
mask = A[:, 0] >= 4            # boolean mask from first column
print(A[mask])                 # rows with first value >=4
print(A[(A[:, 0] >= 4) & (A[:, 2] < 12)])
print(np.where(A > 8, 1, 0))  # thresholded indicators
print(np.percentile(A, [25, 75], axis=0))
print(np.isnan(A).any())
```

NumPy warnings: `&` / `|` for elementwise boolean arrays, with parentheses; `and` / `or` do not operate elementwise. `axis=0` for per-feature summary. `np.std(..., ddof=0)` uses population denominator by default; pandas `Series.std()` defaults to sample `ddof=1`. `reshape(-1,1)` turns a feature vector into a 2D matrix. `np.random.default_rng(42)` offers explicit random generators.

Pandas — DataFrames, indexing, cleaning

```
import pandas as pd
raw = pd.DataFrame({
    'age': [22, 34, None, 45],
    'income': [3200, 4700, 4100, 5200],
    'contract': ['monthly', 'yearly', 'monthly', 'yearly'],
    'churn': [1, 0, 1, 0],
})
print(raw.head(), raw.info(), raw.describe(include='all'))
print(raw.loc[raw['income'] > 4000, ['age', 'contract']])
print(raw.iloc[:2, :2])
print(raw.groupby('contract')['income'].agg(['mean','median','count']))
print(raw['contract'].value_counts(normalize=True))
print(raw.isna().sum())
print(pd.crosstab(raw['contract'], raw['churn'], normalize='index'))
```

Common operations: `.loc` label-based, `.iloc` position-based; `.assign` derived columns; `.merge` joins; `.groupby().agg()` grouped summaries; `.pivot_table` aggregated matrix; `.sort_values`; `.drop_duplicates`; `.astype`; `.to_datetime`; `.str` string extraction; `.dt` datetime features. Avoid chained assignment: `df.loc[mask, 'col'] = value`.

Data joins: Decide cardinality (`one_to_one`, `many_to_one`) and check duplicates before merge. A join that multiplies rows may corrupt target labels or create train-test cross-contamination.

```
# safe join contract when customer_id is unique on the right
# merged = orders.merge(customers, on='customer_id', validate='many_to_one')
```

7. EDA — Exploratory Data Analysis

EDA goal: Understand what exists, what's strange, what could be useful, and what could invalidate the model. EDA is *not* proof that two variables have a causal relationship.

A practical inspection sequence: (1) rows/columns/dtypes, (2) target meaning and balance/distribution, (3) missingness per feature and subgroup, (4) plausible ranges and

inconsistent units, (5) duplicates/IDs/timestamps, (6) single-variable distributions, (7) feature-to-target relationships, (8) feature-to-feature redundancy/correlation, (9) group/time shifts, (10) a written data quality report.

Plot/summary	Good for	Be careful about
Histogram / KDE	Continuous feature shape, skew, modes	Binwidth; KDE smooths evidence
Boxplot	Median, IQR, outlier flags by group	Small groups, distribution details hidden
Scatterplot	Two continuous variables, nonlinearity	Overplotting, correlation ≠ causation
Count/bar plot	Categorical frequency/class imbalance	Denominator matters
Correlation heatmap	Linear redundancy	Pearson ignores nonlinear patterns
Pairplot	A few related features	Expensive/unreadable with many columns
Missingness map	Which columns/rows are incomplete	Missingness may depend on target/group
Time plot	Seasonality, regime changes, drift	Do not use future observations to predict past

Mean vs median: For incomes `[2000, 2200, 2400, 2600, 100000]`, mean is heavily influenced by the last value; median remains 2400. Which is informative depends on the question. **Standard deviation** (écart type) measures spread in original units. **Covariance** depends on feature units; **correlation** standardizes it.

```
import matplotlib.pyplot as plt
import seaborn as sns
# Using a dataframe `train_df` created from training rows only:
# fig, ax = plt.subplots(1, 2, figsize=(10, 4))
# sns.histplot(data=train_df, x='age', kde=True, ax=ax[0])
# sns.boxplot(data=train_df, x='contract', y='income', ax=ax[1])
# plt.tight_layout(); plt.show()
# numeric_corr = train_df.select_dtypes('number').corr(method='spearman')
# sns.heatmap(numeric_corr, annot=True, fmt='.2f'); plt.show()
```

Target-aware EDA: It is fine to inspect target distribution on training rows, but choosing features after seeing test labels leaks information. Group comparisons should reflect base rates and sample counts. For classification report target **prevalence**; for regression inspect scale, skew, zeros and extreme values. For time data inspect ranges, gaps, data collection latency and shifts over time.

Mini EDA report questions: Are values plausible? Could missingness be informative? Is there a class with fewer than 10 samples? Are the same customer IDs repeated? What date is a feature recorded vs the target? Is performance evaluated on the expected population?

8. Missing values, duplicates, inconsistent observations and outliers

Missing values

Common mechanisms: MCAR (missing completely at random), MAR (missing conditional on observed variables), MNAR (missing related to unobserved value). These are assumptions, not labels you can determine perfectly from a missingness plot. Common strategies: drop rows/columns when justified, simple imputation (mean/median/mode), constant category (`Unknown`), missing indicators, iterative/KNN imputation (complexity and leakage considerations).

Always fit imputers on training folds only. Median is robust to extreme observations; mean is efficient for symmetric distributions; most frequent/constant works for categorical inputs. Missingness itself may predict the outcome, so consider `add_indicator=True` and evaluate it. Not every algorithm accepts `NaN` natively; support is model/version-specific.

Duplicates and inconsistencies

Check exact duplicates, entity-level duplicates and accidental one-to-many joins. Normalize category spelling (`'Yearly'`, `'yearly '`) after defining domain rules. Validate measurement units, impossible ages/dates, target labels and overlapping observation windows. Do not indiscriminately drop repeated legitimate events.

Outliers

- **Univariate flags:** IQR rule, z-score for roughly symmetric distributions, robust MAD-based score.
- **Multivariate anomalies:** Isolation Forest, LOF and other density/isolation techniques.
- **Responses:** Verify source error; correct it when supported, cap/winsorize *within the training pipeline*, use robust scaler/loss, transform skewed variables, or retain authentic extremes if prediction of them matters.

Why not remove every flagged point? Extreme customers, expensive homes and rare events may be real, valuable parts of the deployment distribution. Removing them may introduce selection bias.

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
num_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median', add_indicator=True)),
    ('scaler', StandardScaler()),
])
# Fit this inside a full supervised training pipeline, not on X_test.
```

9. Feature engineering, selection and extraction

Feature engineering creates or transforms predictors using knowledge available at the prediction moment: `tenure_months`, `cost_per_use`, weekday, lagged rolling averages, log(1+x) for nonnegative skewed counts, one-hot interactions, bins. Always ask: *could this value be computed for a new case?*

Date example: Prediction today, last purchase 45 days ago → `days_since_last_purchase=45`. Using next week's purchases in a feature would leak the future. For cyclical hour of day, $\sin(2\pi \text{hour}/24)$ and $\cos(2\pi \text{hour}/24)$ keep 23 and 0 close.

Feature selection keeps a subset of original columns:

- **Filter:** variance threshold, mutual information, univariate tests (fit on training fold only).
- **Wrapper:** recursive feature elimination (RFE/RFECV), sequential selection; costly.
- **Embedded:** Lasso coefficients, tree-based importance, regularized linear models.

Feature extraction constructs a lower-dimensional representation: PCA, SVD for sparse text, TF-IDF, embeddings produced from approved classical methods. PCA is unsupervised; it maximizes retained variance, not necessarily target prediction quality.

Multicollinearity: Strong linear relationships among predictors destabilize interpretation of OLS coefficients (large variance), even when predictions remain good. Inspect correlations, VIF under appropriate assumptions, and regularization. Drop one of redundant variables when interpretability requires it.

Text without deep learning: `TfidfVectorizer` transforms documents to token-weighted sparse features; pair with linear SVM or logistic regression for a strong traditional baseline. Fit the vectorizer inside a pipeline during CV.

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
text_model = make_pipeline(TfidfVectorizer(ngram_range=(1, 2)), LogisticRegression(max_iter=1000))
text_model.fit(['great service', 'terrible delay', 'excellent response', 'bad support'], [1, 0, 1,
↪  0])
print(text_model.predict(['excellent support']))  # tiny toy sample; do not trust accuracy
```

10. Encoding, scaling, transformations and imbalance

Categorical encoding

- **OneHotEncoder:** separate binary column for each category, safe for nominal categories; `handle_unknown='ignore'` permits unseen categories but does not imply categories are equivalent; sparse output may increase dimensionality.
- **OrdinalEncoder:** assigns ordered integers; appropriate only when order is real or model/approach intentionally handles category integer codes; do **not** encode `blue=0, red=1, green=2` and assume numeric distances are meaningful.
- **Target/mean encoding:** replaces category with target statistics; must be cross-fitted or leakage-safe within CV, especially for high-cardinality categorical variables. Unseen categories need a fallback.
- **Native categorical modeling:** Some gradient-boosting libraries handle categories specially; check their API and missing/unseen-category behavior.

Scaling and transformations

Method	Formula/idea	Typical use	Caution
StandardScaler	$(x-\mu)/\sigma$	Linear models, KNN, SVM, PCA	Sensitive to large outliers
MinMaxScaler	$(x-\min)/(\max-\min)$	Fixed scale-sensitive algorithms	Held-out values may exceed [0,1]
RobustScaler	$(x-\text{median})/\text{IQR}$	Numeric outliers	Does not delete outliers
Normalizer	Each row divided by its norm	Text/cosine scenarios	**Row** normalization ≠ feature scaling
PowerTransformer	Estimate monotonic power transform	Reduce skew, stabilize variance	Fit training only; data assumptions vary
QuantileTransformer	Remap empirical ranks	Non-normal distributions	Can distort linear distances/extremes
`log1p(x)`	$\log(1+x)$	Nonnegative skew	Negative values require special handling

Why scale? KNN, SVM and PCA are sensitive to feature scales. Regularized linear models also depend on coefficient scale. Trees and tree ensembles split on feature thresholds and are generally invariant to monotonic rescaling of a feature, though numerical and implementation details can change results.

Handling class imbalance

Example: 990 non-churners, 10 churners. Predicting no churn always yields **99% accuracy** but recall for churners is **0**. Options: better evaluation (PR-AUC, positive recall, cost), `class_weight`, threshold tuning on validation data, careful resampling, more minority labels. **Never SMOTE/oversample before splitting/CV**: synthetic neighbors from held-out folds leak into training. Put samplers inside `imblearn.pipeline.Pipeline` where they run on training folds only. Evaluate probabilities/calibration carefully after sampling; resampled class balance may alter probability interpretation.

```
# Optional package: imbalanced-learn
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
balanced_model = ImbPipeline([
    ('scale', StandardScaler()),
    ('smote', SMOTE(random_state=42)),
    ('clf', LogisticRegression(max_iter=1000)),
])
# Cross-validate balanced_model on X_train, y_train; never resample X_test.
```

If there are categorical variables, regular SMOTE on one-hot features can create implausible pseudo-categories; consider SMOTENC or class weights as appropriate.

11. Splitting, stratification, cross-validation and leakage

Holdout: `train_test_split(..., test_size=.2, random_state=42)` gives a fixed random split. For classification, `stratify=y` roughly preserves class proportions (when class counts permit); this does **not** prevent entity or time leakage.

Three roles: Training learns parameters. Validation/CV chooses preprocessing, model, hyperparameters and decision threshold. Test is the last held-out audit, ideally used once. On small datasets, reserve test and use cross-validation within the development set; nested CV gives a less biased estimate of an entire tuning procedure when data are limited.

Dataset structure	Correct approach	Why
IID rows and classification	StratifiedKFold	Maintain label proportions across folds
Same customer/patient repeated	GroupKFold / StratifiedGroupKFold	Prevent group overlap across train/validation
Predicting future	Chronological split / TimeSeriesSplit	Prevent future→past leakage
Spatially correlated sites	Spatial/group holdout	Test generalization to locations
Highly rare positive labels	Stratification where feasible; report variability	Some folds may otherwise contain no positives

Time-series detail: `TimeSeriesSplit` does not automatically enforce an appropriate gap or solve all label-overlap issues; choose an embargo/gap for the specific forecast horizon. Feature engineering for lags/rolling windows must use only earlier timestamps. Do not randomly shuffle temporal forecasting observations.

Example — leakage-free mixed data:

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split, cross_val_score

df = pd.DataFrame({
    'age': [20, 40, 31, 55, 35, 47, 29, 61, 22, 38, 44, 56],
    'income': [2, 5, 3, 6, 4, None, 2, 7, 3, 4, 5, 6],
    'contract': ['m', 'y'] * 6,
    'churn': [1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0],
})
X, y = df.drop(columns='churn'), df['churn']
X_dev, X_test, y_dev, y_test = train_test_split(
    X, y, stratify=y, random_state=42, test_size=.25)
num = Pipeline([('imp', SimpleImputer(strategy='median')), ('sc', StandardScaler())])
cat = Pipeline([('imp', SimpleImputer(strategy='most_frequent')),
                ('ohe', OneHotEncoder(handle_unknown='ignore'))])
pre = ColumnTransformer([('num', num, ['age', 'income']), ('cat', cat, ['contract'])])
pipe = Pipeline([('pre', pre), ('model', LogisticRegression(max_iter=1000))])
# This dataset is deliberately tiny, so a real CV estimate would be unstable.
pipe.fit(X_dev, y_dev)
print('Held-out predictions:', pipe.predict(X_test))
```

Always audit: Were predictions on exactly unseen entities/times? Were preprocessors, encoders, samplers and feature selection fit inside folds? Was the final threshold chosen on test labels? Did we tune repeatedly after seeing test scores? Did any column reveal the outcome by construction?

Practice challenge: Build a churn dataset with two numeric and two categorical columns, intentional missingness, and a rare positive class. Fit a `ColumnTransformer` pipeline; compare logistic regression and random forest by stratified cross-validation; explain one case where random CV would still be invalid. See Chapter 52.

Part III — Regression Models

Chapters 12–21 · Predict continuous values. Example targets: house price, delivery time, electricity demand, temperature. Keep a **DummyRegressor** baseline; choose metrics that match the target scale and cost of errors.

A common regression experiment: reusable pattern

For each algorithm: define a development split, fit inside a pipeline, compare with cross-validation, select using a predeclared metric, then report holdout results. Below is a minimal baseline using built-in data; later chapters swap the estimator.

```
from sklearn.datasets import load_diabetes
from sklearn.dummy import DummyRegressor
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score
X, y = load_diabetes(return_X_y=True)
X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=.2, random_state=42)
base = DummyRegressor(strategy='mean').fit(X_dev, y_dev)
model = make_pipeline(StandardScaler(), Ridge(alpha=1.0))
cv_mae = -cross_val_score(model, X_dev, y_dev, cv=5, scoring='neg_mean_absolute_error')
model.fit(X_dev, y_dev)
pred = model.predict(X_test)
print('Dummy MAE:', mean_absolute_error(y_test, base.predict(X_test)))
print('CV MAE:', cv_mae.mean(), cv_mae.std())
print('Test MAE/RMSE/R2:', mean_absolute_error(y_test, pred),
      root_mean_squared_error(y_test, pred), r2_score(y_test, pred))
```

Note: `root_mean_squared_error` is a modern scikit-learn API; for older installed versions use `np.sqrt(mean_squared_error(y_test, pred))`. Built-in educational datasets do not imply meaningful real-world decisions.

12. Simple Linear Regression

Idea: Fit a straight line to map one feature x to a continuous target y:

$$\hat{y} = \beta_0 + \beta_1 x.$$

β_0 is intercept: predicted y when $x = 0$ (may be outside observed range); β_1 is expected prediction change for one additional unit of x, holding other things fixed where applicable. This is *association* in observational data, not causal proof.

Training: Ordinary least squares minimizes sum of squared residuals $\sum_i (y_i - \hat{y}_i)^2$. For a single predictor with variation, $\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$ and $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$. Residual $e_i = y_i - \hat{y}_i$.

```
import numpy as np
from sklearn.linear_model import LinearRegression
X = np.array([1.,2.,3.,4.,5.]).reshape(-1, 1)
y = np.array([2.,4.,5.,4.,5.])
model = LinearRegression().fit(X, y)
print('intercept:', model.intercept_, 'slope:', model.coef_[0])
print('prediction for x=6:', model.predict([[6.]])[0])
```

Interpretation: slope of 2.5 in a house-size→price model measured in $1000 per m² means +1 m² predicts +$2500 *according to that fitted linear association*, not a guaranteed causal gain. **Diagnostics:** residual vs fitted (curvature, heteroscedasticity), residual vs time (autocorrelation), influential high-leverage points; inspect performance on held-out data. **Assumptions for valid OLS inference:** linear conditional mean, independent errors according to design, appropriate exogeneity; constant variance and normal errors matter for some classical inference, not for simply computing a line. Strong extrapolation is unsafe.

Hyperparameters: `fit_intercept`, `positive` (specific implementations); no tree-depth or learning-rate parameter. **Strength:** interpretable fast baseline. **Weakness:** nonlinear patterns, outliers and multicollinearity can degrade performance or interpretation.

Oral question: How does the regression slope differ from correlation? *Slope is unit-dependent change in predicted target per predictor unit; correlation is unitless standardized linear association.*

13. Multiple Linear Regression

Idea: Predict from multiple inputs:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p = \beta_0 + X\beta.$$

Each β_j represents the fitted change for x_j **holding included other features constant**; correlation among inputs can make that interpretation unstable. Under full rank, OLS normal equations give $\hat{\beta} = (X^\top X)^{-1} X^\top y$ (intercept handled appropriately), but actual implementations use more numerically stable methods (SVD/QR), and ordinary inverse need not exist.

```
import pandas as pd
from sklearn.linear_model import LinearRegression
train = pd.DataFrame({'size':[40,60,80,100,120], 'rooms':[1,2,2,3,4],
                      'price':[80,119,149,195,242]})
X, y = train[['size','rooms']], train['price']
reg = LinearRegression().fit(X, y)
print(dict(zip(X.columns, reg.coef_)), reg.intercept_)
print(reg.predict(pd.DataFrame({'size':[90], 'rooms':[3]})))
```

Check: numeric inputs, enough independent observations, sensible data types, high collinearity/VIF, residual patterns and out-of-sample performance. `n_features` close to `n_samples` may overfit; use Ridge. **Categorical predictors** need encoding; **interactions** allow effect of one predictor to depend on another. **Soutenance:** Why can adding features increase training R^2 without improving validation? *More parameters cannot worsen in-sample OLS fit but can learn noise.*

14. Polynomial Regression

Idea: A linear model in *engineered nonlinear features*; for one feature:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_d x^d.$$

Polynomial regression is still linear *in coefficients*. Degree 2 curves, degree 3 permits a changing inflection pattern; high degrees can oscillate and explode outside the data range. For many input features, interactions expand feature count quickly. Use `PolynomialFeatures` inside a pipeline, scale/regularize, validate degree.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import Ridge
poly = make_pipeline(PolynomialFeatures(degree=2, include_bias=False),
                     StandardScaler(), Ridge(alpha=1.0))
poly.fit(X, y)   # X, y defined above; illustrative tiny data
print(poly.predict(pd.DataFrame({'size':[90], 'rooms':[3]})))
```

Hyperparameters: degree, interaction_only, include_bias, regularization. **Trap:** Adding polynomial expansion *before splitting/CV* can be okay only if it is stateless, but all learned transformations (scaler, feature selection) must be fit within folds; using a pipeline is the robust default. **Oral:** What happens as degree increases? *Training flexibility rises; validation error may eventually rise due to variance.*

15. Ridge, Lasso and Elastic Net

All are linear models with coefficient penalties that often improve generalization and stability. Standardize numeric inputs first; otherwise penalties act unevenly across differently scaled features.

Method	Minimized objective (schematic)	Effect	`sklearn` estimator
OLS	$\sum(y-\hat{y})^2$	No coefficient penalty	`LinearRegression`

Method	Minimized objective (schematic)	Effect	`sklearn` estimator
Ridge (L2)	$\sum(y-\hat{y})^2+\alpha\sum_j\beta_j^2$	Shrinks coefficients; usually not exactly zero	`Ridge`
Lasso (L1)	$\frac{1}{2n}\sum(y-\hat{y})^2+\alpha\sum_j\lvert\beta_j\rvert$	Can set coefficients to zero	`Lasso`
Elastic Net	$\frac{1}{2n}\sum(y-\hat{y})^2+\alpha[\rho\lVert\beta\rVert_1+(1-\rho)\lVert\beta\rVert_2^2/2]$	Combines sparsity and correlated-feature stabilization	`ElasticNet`

The objective normalization varies by estimator; **do not compare raw alpha values across models as directly equivalent**. Intercept is typically excluded from the penalty. `alpha→0` reduces regularization (subject to numerical considerations). Larger `alpha` strengthens regularization. `l1_ratio` in Elastic Net is ρ: near 1 resembles Lasso, near 0 resembles Ridge.

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
models = {'ridge': Ridge(alpha=1.0),
          'lasso': Lasso(alpha=0.1, max_iter=10000),
          'elastic': ElasticNet(alpha=0.1, l1_ratio=0.5, max_iter=10000)}
for name, estimator in models.items():
    fitted = make_pipeline(StandardScaler(), estimator).fit(X, y)
    print(name, fitted.predict(X)[:2])  # training-only demo, not evaluation
```

When choose: Ridge when many modest/correlated effects; Lasso for sparse interpretation (but selections can be unstable with correlated inputs); Elastic Net when you want a balance. **Limits:** still linear in transformed features, need tuning, penalized coefficients are biased by design. **CV:** `RidgeCV`/`LassoCV` handle some selection workflows, but avoid leakage from global preprocessing; wrap preprocessing properly. **Oral:** Why might Lasso choose one feature arbitrarily from two strongly correlated features? *Several sparse solutions may fit almost equally well.*

16. K-Nearest Neighbors Regression

Idea: Store training samples. For a new row, find k nearby rows according to chosen distance, average their targets (optionally weighted inversely by distance). Training is cheap; inference can be costly. It is nonparametric: no fixed global linear formula.

$$\hat{y}(x)=\frac{1}{k}\sum_{i\in N_k(x)}y_i \quad \text{(uniform weights).}$$

```
from sklearn.neighbors import KNeighborsRegressor
knn = make_pipeline(StandardScaler(), KNeighborsRegressor(n_neighbors=5, weights='distance'))
knn.fit(X_dev, y_dev)
print(knn.predict(X_test[:3]))
```

Hyperparameters: `n_neighbors`, `weights`, `metric`, `p` for Minkowski. Small k → flexible/possibly noisy; large k → smoother/possibly biased. **Must scale** meaningful numeric distances; irrelevant/high-dimensional features hurt (curse of dimensionality). **Does not naturally extrapolate** beyond observed targets. Missing/categorical inputs need preprocessing. **Soutenance:** Why can income dominate age? *Euclidean distances depend on numeric units; a wide-scale feature can dominate.*

17. Decision Tree Regression

Idea: Recursively split feature space using thresholds to reduce target impurity (e.g., squared error), and predict a constant (often mean target) in each terminal leaf. Captures interactions/nonlinear rules without feature scaling. Easy to visualize at shallow depth, unstable when deep.

Split concept: choose feature and threshold that yield greatest reduction in weighted squared errors. A regression tree is **piecewise constant** and ordinarily does not extrapolate linearly.

```
from sklearn.tree import DecisionTreeRegressor, plot_tree
import matplotlib.pyplot as plt
tr = DecisionTreeRegressor(max_depth=3, min_samples_leaf=5, random_state=42)
tr.fit(X_dev, y_dev)
print(tr.predict(X_test[:3]))
plt.figure(figsize=(12, 6)); plot_tree(tr, max_depth=2, filled=True); plt.show()
```

Hyperparameters: `max_depth`, `min_samples_leaf`, `min_samples_split`, `max_leaf_nodes`, `ccp_alpha`, `max_features`. **Overfitting:** fully grown trees isolate noise; prune or constrain. **Soutenance:** Why do trees not need scaling? *Monotonic rescaling mostly preserves ordering and potential split partitions.*

18. Random Forest and Extra Trees Regression

Random Forest (bagging): Fit many different trees on bootstrap samples (`bootstrap=True` normally) with random subsets of features at splits; aggregate predictions (mean). Averaging imperfectly correlated trees often lowers variance vs a single tree. **Out-of-bag** (OOB) predictions can provide an internal approximate evaluation when bootstrap is enabled, but do not replace a proper independent test for every design.

Extra Trees (Extremely Randomized Trees): Typically choose more random split thresholds than standard forests, introducing extra randomness; scikit-learn ExtraTrees use `bootstrap=False` by default. They can be faster and low variance, but depend on problem and tuning.

```
from sklearn.ensemble import RandomForestRegressor, ExtraTreesRegressor
rf = RandomForestRegressor(n_estimators=300, min_samples_leaf=3,
                           max_features=1.0, random_state=42, n_jobs=-1)
et = ExtraTreesRegressor(n_estimators=300, min_samples_leaf=3,
                         random_state=42, n_jobs=-1)
for m in (rf, et):
    m.fit(X_dev, y_dev)
    print(type(m).__name__, mean_absolute_error(y_test, m.predict(X_test)))
```

Hyperparameters: number of trees, depth, minimum leaf size, feature subsampling, bootstrap, parallelism. More trees mainly stabilize average rather than cure fundamentally biased splits. **Limitations:** less extrapolation; large models; naive impurity feature importances can prefer high-cardinality/continuous columns; permutation importance on held-out data is usually more informative. **OOB:** use `oob_score=True` only when supported with `bootstrap=True`.

Oral: Why does bagging reduce variance? *Sampling and randomized splits create diverse estimators, and averaging cancels some independent errors.*

19. Support Vector Regression (SVR)

Idea: Find a function approximately flat with respect to model complexity, tolerating prediction errors smaller than an ϵ margin; points outside the epsilon tube incur penalty. Linear or nonlinear kernel mapping can represent complex shapes.

$$L_\epsilon(y, \hat{y}) = \max(0, |y - \hat{y}| - \epsilon).$$

Key controls: `C` is penalty for violations (larger C fits more strongly), `epsilon` is tube width, `gamma` controls locality of RBF kernel (`gamma='scale'` practical baseline), `kernel` chooses linear/polynomial/RBF. Scaling is essential; kernel SVR can be slow for large n; `LinearSVR` scales better for linear settings.

```
from sklearn.svm import SVR
svr = make_pipeline(StandardScaler(), SVR(kernel='rbf', C=10.0, epsilon=0.1))
svr.fit(X_dev, y_dev)
print(mean_absolute_error(y_test, svr.predict(X_test)))
```

Why use: Moderate-sized, reasonably scaled dataset with nonlinear dependencies. **Why avoid:** Millions of rows with RBF kernel, poor feature scaling, need for transparent coefficients or strong extrapolation. **Oral:** What does epsilon mean? *An error tolerance region around prediction within which deviations are not penalized by the epsilon-insensitive loss.*

20. Boosted-tree regression: Gradient Boosting, AdaBoost, XGBoost, LightGBM, CatBoost

Boosting builds an additive ensemble **sequentially**, each stage addressing weaknesses of the current predictor. Unlike a random forest (largely independent trees averaged), later boosting stages depend on earlier errors or gradients. Typical prediction: $F_M(x) = F_0(x) + \sum_{m=1}^{M} \eta h_m(x)$, with learning rate η. This is schematic; exact implementations vary.

GradientBoostingRegressor

Fits successive trees to loss-function gradients (for squared error, related to residuals). Tune `n_estimators`, `learning_rate`, `max_depth` or `max_leaf_nodes`, `subsample`, `min_samples_leaf`. Smaller learning rate often requires more trees. Classical gradient boosting may be slower than histogram-based alternatives.

AdaBoostRegressor

Focuses successive weak learners on difficult observations via weighting procedures (algorithm differs from gradient boosting); can be sensitive to noisy/outlying targets. Tune `n_estimators`, `learning_rate`, and weak `estimator`.

HistGradientBoostingRegressor

Scikit-learn histogram-binned gradient boosting, efficient for many tabular problems and supports missing values. Tune `max_iter`, `learning_rate`, `max_leaf_nodes`, `l2_regularization`, early stopping. Check implementation categorical support requirements.

```
from sklearn.ensemble import (GradientBoostingRegressor, AdaBoostRegressor,
                              HistGradientBoostingRegressor)
candidates = [GradientBoostingRegressor(n_estimators=150, learning_rate=.05, random_state=42),
              AdaBoostRegressor(n_estimators=100, random_state=42),
              HistGradientBoostingRegressor(max_iter=150, random_state=42)]
for m in candidates:
    m.fit(X_dev, y_dev)
    print(type(m).__name__, mean_absolute_error(y_test, m.predict(X_test)))
```

External libraries

Library	Central idea	High-value controls	Advantages/cautions
XGBoost	Regularized gradient-boosted trees, efficient split finding	`n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`, `reg_lambda`, `tree_method`	Powerful tabular baseline; validate overfitting
LightGBM	Histogram + leaf-wise growth, efficient categorical/histogram workflows	`num_leaves`, `max_depth`, `min_child_samples`, `learning_rate`, `n_estimators`	Fast on large tabular data; can overfit small samples
CatBoost	Ordered boosting and categorical-feature handling	`iterations`, `depth`, `learning_rate`, `l2_leaf_reg`, `cat_features`	Useful with many categoricals; keep category semantics explicit

```
# Optional dependencies: pip install xgboost lightgbm catboost
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
from catboost import CatBoostRegressor
```

```
boosters = [
    XGBRegressor(n_estimators=150, max_depth=4, learning_rate=.05,
                 objective='reg:squarederror', random_state=42, n_jobs=4),
    LGBMRegressor(n_estimators=150, num_leaves=15, learning_rate=.05,
                  random_state=42, verbosity=-1),
    CatBoostRegressor(iterations=150, depth=4, learning_rate=.05,
                      verbose=False, random_seed=42),
]
for m in boosters:
    m.fit(X_dev, y_dev)
    print(type(m).__name__, mean_absolute_error(y_test, m.predict(X_test)))
```

Categorical data caveat: This demo uses numeric arrays. To use CatBoost native categories, pass a DataFrame with appropriately specified `cat_features`; XGBoost/LightGBM categorical support differs; consult current documentation. **Early stopping:** Use a *training-internal validation portion*; never supply final test labels in `eval_set` for training decisions. **Why boosted models work:** Additive corrections create flexible nonlinear functions; regularization, shrinkage, sampling, depth and stopping control variance.

Soutenance compare: Random Forest trains randomized trees largely independently and averages them; boosting is sequential additive optimization. One does not universally dominate the other.

21. Regression metrics and diagnostics

Let $e_i = y_i - \hat{y}_i$ and $\bar{y}$ the mean of *true* evaluation targets. A metric must reflect the costs/units of the actual problem.

Metric	Definition	Direction	When useful / limitation
MAE	$\frac{1}{n}\sum \lvert e_i \rvert$	↓ better	In target units, robust-ish to outliers, interpretable
MSE	$\frac{1}{n}\sum e_i^2$	↓	Squared units; strongly punishes large errors
RMSE	$\sqrt{\frac{1}{n}\sum e_i^2}$	↓	Target units, sensitive to large errors
Median AE	$\text{median}(\lvert e_i \rvert)$	↓	Typical absolute error, ignores tail failures
R^2	$1 - \frac{\sum e_i^2}{\sum (y_i - \bar{y})^2}$	↑	Compared with predicting mean **of the evaluation set**, can be negative
Adjusted R^2	$1 - (1 - R^2)\frac{n-1}{n-p-1}$	↑	Penalizes predictors; $n > p+1$; classical OLS context
MAPE	$\frac{1}{n}\sum \lvert e_i / y_i \rvert$	↓	Relative error; undefined or unstable at zero/near-zero labels
sMAPE	Common forms use $2\lvert y - \hat{y} \rvert / (\lvert y \rvert + \lvert \hat{y} \rvert)$	↓	Convention varies; undefined when both zero
RMSLE	Root MSE of `log1p(y)` and `log1p(pred)`	↓	Nonnegative/compatible values; emphasizes relative scale
Huber / pinball	Piecewise robust / quantile loss	↓	Robustness, asymmetric costs, prediction intervals

Worked example: true `[3, -0.5, 2, 7]`, prediction `[2.5, 0, 2, 8]`: errors `[0.5, -0.5, 0, -1]`; absolute errors `[.5,.5,0,1]`; MAE $= .5$, MSE $= (.25 + .25 + 0 + 1)/4 = .375$, RMSE $\approx .612$. Avoid MAPE here because a true value is negative and ratio errors are hard to interpret; near-zero truths make MAPE especially problematic.

```
import numpy as np
from sklearn.metrics import (mean_absolute_error, mean_squared_error,
                             root_mean_squared_error, r2_score,
                             median_absolute_error)
y_true = np.array([3., -.5, 2., 7.]); y_pred = np.array([2.5, 0., 2., 8.])
print('MAE:', mean_absolute_error(y_true,y_pred))
print('MSE:', mean_squared_error(y_true,y_pred))
print('RMSE:', root_mean_squared_error(y_true,y_pred))
print('R2:', r2_score(y_true,y_pred))
print('Median AE:', median_absolute_error(y_true,y_pred))
```

CV scoring trap: scikit-learn `scoring='neg_mean_absolute_error'` returns negative MAE because scorer conventions maximize values; negate to display MAE. **R^2 caveat:** R^2 is not "percent predictions correct" and may be misleading for distribution shift; do not compare R^2 across unlike targets as a universal goodness score. **Error inspection:** Plot residuals vs predictions, vs time, and vs key groups; compare MAE across segments and rare extremes. If a $1000 error on a cheap home differs from that on an expensive one, use domain-aware metrics too.

Diagnose failure: Is the model worse on the newest dates? Does it systematically overpredict small targets? Do prediction intervals cover enough outcomes? Would constant-mean `DummyRegressor` perform nearly as well? Are extreme errors operationally unacceptable?

Regression decision card

- Need clear coefficient story, stable linear relationship → **Linear / Ridge** (with preprocessing).
- Need nonlinear small/medium data, interpretable thresholds → **small decision tree**.
- Need solid nonlinear tabular baseline with fewer tuning demands → **Random Forest / Extra Trees**.
- Need strong predictive accuracy on structured tabular data → **boosted trees** with proper CV.
- Need local smooth interpolation on low-dimensional scaled features → **KNN**.
- Need nonlinear smooth function on modest sample size → **SVR (RBF)**.
- Need feature sparsity → **Lasso / ElasticNet**.

Chapter 21 oral challenge: If MAE decreases but RMSE increases, what likely changed? *Average absolute errors fell but one or more large misses increased; RMSE emphasizes large errors.*

Part IV — Classification Models

Chapters 22–32 · Predict a discrete class, optionally with estimated class probabilities. Typical binary questions: "Will the customer churn?"; multiclass: "Which category?"; multilabel: "Which tags apply?". Learn the class frequencies and error costs before choosing a metric.

How to compare classifiers fairly

Keep preprocessing/splitting constant. Establish a `DummyClassifier(strategy='most_frequent')` and optionally `strategy='prior'` probabilistic baseline. Compare logistic regression, random forest and a boosted tree by cross-validation on the same training data, with *one clearly defined primary metric* and secondary metrics.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_validate
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.dummy import DummyClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
X, y = load_breast_cancer(return_X_y=True)
X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=.2,
    stratify=y, random_state=42)
```

```
models = {
    'dummy': DummyClassifier(strategy='prior'),
    'logistic': make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)),
    'forest': RandomForestClassifier(n_estimators=200, random_state=42),
}
for name, pipe in models.items():
    scores = cross_validate(pipe, X_dev, y_dev, cv=5,
                            scoring=['balanced_accuracy', 'roc_auc'])
    print(name, scores['test_balanced_accuracy'].mean(), scores['test_roc_auc'].mean())
# For a real project, choose using CV; fit the selected workflow once on X_dev.
chosen = models['logistic'].fit(X_dev, y_dev)
print(confusion_matrix(y_test, chosen.predict(X_test)))
print(classification_report(y_test, chosen.predict(X_test)))
```

This built-in dataset is educational. Its class meaning should not be used to make medical decisions; verify which class is positive in every new problem.

22. Logistic Regression

Despite the name, logistic regression is a classifier. It first computes a linear score (logit) $z = w^\top x + b$, then uses sigmoid for a binary probability:

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad P(y=1 \mid x) = \sigma(z).$$

The **log-odds** are linear: $\log \frac{p}{1-p} = w^\top x + b$. A coefficient w_j adds w_j to log-odds for a one-unit rise in feature j (holding other features fixed); e^{w_j} multiplies the *odds*, not the probability. A fixed threshold of 0.5 turns p into class prediction; 0.5 is not universally optimal.

Loss: binary cross-entropy / log loss $-\frac{1}{n}\sum_i [y_i \log p_i + (1-y_i)\log(1-p_i)]$, optionally with regularization. This penalizes confidently incorrect predictions more than mildly incorrect ones. Multi-class logistic regression uses softmax/multinomial loss (solver/configuration support varies by library version).

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
pipe = make_pipeline(StandardScaler(), LogisticRegression(C=1.0, max_iter=2000,
                                                          class_weight=None))
pipe.fit(X_dev, y_dev)
prob = pipe.predict_proba(X_test)[:, 1]
pred = (prob >= .5).astype(int)
print(prob[:5], pred[:5])
```

Hyperparameters: `C` inverse regularization strength (smaller → stronger regularization), `penalty` (where supported), `solver`, `class_weight`, `max_iter`. **Scaling** helpful; interpretable conditional *log odds* after encoding; interaction/nonlinear features possible. **Assumptions:** a linear log-odds relationship in chosen features, meaningful labels, and a valid sample/split. Inputs do not need to follow a Gaussian distribution. **Soutenance:** Does a coefficient of 0.7 mean a 70% higher probability? *No; odds multiply by* $e^{0.7} \approx 2.01$.

23. K-Nearest Neighbors (KNN) classification

Idea: Find k nearest labeled training observations, predict by majority vote or distance-weighted vote. Estimated `predict_proba` is based on class frequencies/weights among neighbors, not necessarily well-calibrated probability. **No explicit equation is learned globally;** inference stores/searches training data.

```
from sklearn.neighbors import KNeighborsClassifier
knn = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=7,
                                                           weights='distance'))
knn.fit(X_dev, y_dev)
print(knn.predict(X_test[:5]), knn.predict_proba(X_test[:2]))
```

Hyperparameters: k, metric, distance weights, search algorithm. **Small k** can overfit/noise; **large k** can overlook minority regions. Feature scaling is essential for distance-based KNN. High-dimensional data dilute distance relevance; irrelevant features and class imbalance are serious

problems. **Oral:** Is KNN a lazy learner? *Yes: work happens mostly at prediction; it stores examples instead of fitting a compact parametric decision rule.*

24. Naive Bayes (Gaussian, Multinomial, Bernoulli)

Idea: Bayes rule plus conditional feature independence given the class:

$$P(y = c \mid x) \propto P(y = c) \prod_{j=1}^{p} P(x_j \mid y = c).$$

Multiplications are evaluated in log-space in practice to avoid underflow. "Naive" independence is often false, but classification can still work well. **GaussianNB** assumes each numeric feature is normally distributed *within each class*; **MultinomialNB** suits nonnegative count/term-frequency-like features; **BernoulliNB** suits binary occurrence/absence features. Do not feed negative standardized features to MultinomialNB.

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.pipeline import make_pipeline
nb_numeric = GaussianNB().fit(X_dev, y_dev)
print(nb_numeric.predict(X_test[:3]))
text_nb = make_pipeline(CountVectorizer(), MultinomialNB(alpha=1.0))
text_nb.fit(['good service', 'very good', 'bad delay', 'terrible delay'], [1,1,0,0])
print(text_nb.predict(['good response']))
# BernoulliNB supports binary feature matrices.
```

Hyperparameters: smoothing `alpha`, Gaussian `var_smoothing`, class priors. **Pros:** fast, strong sparse-text baseline, works with small datasets. **Limits:** correlated features double-count evidence, probabilities can be poorly calibrated. **Oral:** How does smoothing help? *Avoids zero likelihood for unseen feature values/events.*

25. Decision Tree Classifier

Idea: Recursively ask threshold/category questions to partition data; each leaf predicts the majority class and usually provides empirical class proportions. For class proportions p_k:

$$\text{Gini} = 1 - \sum_k p_k^2, \qquad H = -\sum_k p_k \log_2 p_k.$$

A split is useful when weighted child impurity falls relative to parent. Entropy and Gini often yield similar but not identical trees. A perfect training tree may be overfit; impurity is *not* predictive accuracy by itself.

```
from sklearn.tree import DecisionTreeClassifier, export_text
tr = DecisionTreeClassifier(max_depth=4, min_samples_leaf=5, random_state=42)
tr.fit(X_dev, y_dev)
print(export_text(tr, max_depth=2))
```

Hyperparameters: `criterion='gini'/'entropy'/'log_loss'`, `max_depth`, `min_samples_leaf`, `ccp_alpha`, `class_weight`. **Strengths:** nonlinear interactions, explainable thresholds, little need to scale. **Weaknesses:** unstable split selection, high variance and discontinuous probabilities. **Oral:** Why does a tree overfit? *Repeated branching can isolate tiny training groups and memorize noise.*

26. Random Forest and Extra Trees Classifier

Random Forest: Bootstrap training samples + randomized feature selection, average many tree votes/probabilities. **Extra Trees:** Randomized candidate thresholds usually add variance-reducing diversity; sklearn default `bootstrap=False`. Aggregation improves robustness relative to one unrestricted tree but does not guarantee calibration or freedom from data leakage.

```
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
for est in [RandomForestClassifier(n_estimators=250, min_samples_leaf=2,
                                   class_weight='balanced', random_state=42),
            ExtraTreesClassifier(n_estimators=250, min_samples_leaf=2,
                                 random_state=42)]:
    est.fit(X_dev, y_dev)
    print(type(est).__name__, est.score(X_test, y_test))
```

Hyperparameters: trees, depth, feature subsampling, min leaf size, `class_weight`, bootstrap. **Good for:** nonlinearity, numeric and encoded categorical features, fairly reliable baseline. **Limitations:** class probability estimates may need calibration, correlated feature importance unstable, high-cardinality bias in impurity importance, limited ability to extrapolate trends. **Oral:** What separates bagging from boosting? *Bagging creates diverse learners and aggregates; boosting builds sequential corrections.*

27. Support Vector Machines (SVM)

Idea: Find a decision boundary with large margin between classes, penalizing points within margin or misclassified points. In a linear SVM, the decision rule is $\text{sign}(w^\top x + b)$; support vectors define the effective boundary. **Kernel trick** computes similarities to represent a nonlinear boundary without explicitly forming the transformed features.

Hinge loss: $\max(0, 1 - yf(x))$ for labels encoded $\{-1, +1\}$. A large `C` penalizes margin violations more strongly, potentially overfitting. For RBF kernel, large `gamma` makes influence more local and highly flexible; small `gamma` creates smoother boundaries. Scaling is essential.

```
from sklearn.svm import SVC, LinearSVC
rbf = make_pipeline(StandardScaler(), SVC(kernel='rbf', C=2, gamma='scale'))
rbf.fit(X_dev, y_dev)
print('margin scores:', rbf.decision_function(X_test[:3]))
# SVC does not provide predict_proba unless probability=True;
# probability=True adds a probability calibration procedure and cost.
```

When choose: small/medium-sized data, high-dimensional sparse text (often `LinearSVC`) or complex scaled feature boundaries (RBF). **Limits:** nonlinear SVC training scales poorly with sample count, opaque decisions, uncalibrated raw decision scores. **Oral:** What's the difference between margin and probability? *Margin is distance-related decision confidence, not inherently calibrated P(class).*

28. Linear Discriminant Analysis (LDA) and Quadratic Discriminant Analysis (QDA)

Both are **generative classifiers** approximating each class distribution as Gaussian. LDA assumes a **shared covariance matrix** across classes → linear decision boundaries; QDA estimates **separate class covariance** → quadratic boundaries. Under their assumptions, Bayes rule gives class predictions and probabilities.

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis, QuadraticDiscriminantAnalysis
for model in (LinearDiscriminantAnalysis(), QuadraticDiscriminantAnalysis(reg_param=.1)):
    model.fit(X_dev, y_dev)
    print(type(model).__name__, model.score(X_test, y_test))
```

Tradeoffs: LDA fewer covariance parameters and often more stable on smaller datasets; QDA more flexible but needs more data and can have singular covariance. Consider scaling/regularization based on data, and cross-validation for assumptions. **LDA dimensionality reduction** also exists (supervised separation); it is not the same as PCA (unsupervised variance). **Oral:** If covariances differ between classes, what changes? *QDA allows class-specific covariance and curved boundaries, but estimates many more parameters.*

29. Boosting classifiers: AdaBoost, Gradient Boosting, XGBoost, LightGBM and CatBoost

Core idea: Train successive weak learners to improve a combined model. Gradient boosting optimizes a differentiable classification loss (often log loss) through additive stagewise updates. AdaBoost adaptively weights hard training instances. Boosted trees can be extremely strong for structured data but must be regularized and validated.

```
from sklearn.ensemble import (AdaBoostClassifier, GradientBoostingClassifier,
                              HistGradientBoostingClassifier)
models = [AdaBoostClassifier(n_estimators=100, random_state=42),
          GradientBoostingClassifier(n_estimators=150, learning_rate=.05,
                                     max_depth=2, random_state=42),
          HistGradientBoostingClassifier(max_iter=150, learning_rate=.05,
                                         random_state=42)]
for est in models:
    est.fit(X_dev, y_dev)
```

```
    print(type(est).__name__, roc_auc_score(y_test, est.predict_proba(X_test)[:, 1]))
```

External boosted classifiers (optional):

```
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier
candidates = [
  XGBClassifier(n_estimators=200, max_depth=3, learning_rate=.05,
                eval_metric='logloss', random_state=42, n_jobs=4),
  LGBMClassifier(n_estimators=200, num_leaves=15, learning_rate=.05,
                 random_state=42, verbosity=-1),
  CatBoostClassifier(iterations=200, depth=4, learning_rate=.05,
                     random_seed=42, verbose=False),
]
for est in candidates:
    est.fit(X_dev, y_dev)
    print(type(est).__name__, roc_auc_score(y_test, est.predict_proba(X_test)[:, 1]))
```

What's different? XGBoost emphasizes regularized, optimized boosting; LightGBM histogram/leaf-wise techniques can accelerate training, especially on larger data; CatBoost has distinctive ordered categorical processing. **Main hyperparameters:** iterations, depth/leaves, learning rate, row/column subsampling, minimum leaf size and L1/L2 constraints. Use class weighting/imbalance options thoughtfully; they alter the training objective and may affect calibration. **Early stopping must use a development validation set, not the final test set.**

Soutenance: Why choose XGBoost over logistic regression? *For validated nonlinear interactions and stronger out-of-sample metrics; logistic remains a strong, fast and interpretable baseline.*

30. SGD-based linear classifiers

Stochastic Gradient Descent iteratively optimizes a chosen loss over training examples/mini-batches. `SGDClassifier` supports linear separators with hinge (linear SVM-like), logistic (`loss='log_loss'`) and other losses, often good for huge or sparse data. It is an *optimizer-based estimator family*, not a single decision-boundary shape.

```
from sklearn.linear_model import SGDClassifier
sgd = make_pipeline(StandardScaler(), SGDClassifier(loss='log_loss', alpha=.0001,
                                                    max_iter=2000, random_state=42))
sgd.fit(X_dev, y_dev)
print(sgd.predict_proba(X_test[:3]))
```

Hyperparameters: `loss`, `alpha` regularization, `learning_rate`, `eta0`, `max_iter`, `class_weight`, `average`. **Online training:** some estimators offer `partial_fit`, but you must coordinate preprocessing updates and avoid hidden state contamination. **Important:** standardization helps optimization; early stopping, data shuffling and learning-rate tuning affect convergence. **Oral:** Difference between logistic regression and SGDClassifier with log loss? *Both can learn linear logistic predictors; SGDClassifier uses stochastic optimization and has different defaults/convergence behavior.*

31. Binary, multiclass and multilabel classification

- **Binary:** one of two mutually exclusive classes, e.g. churn yes/no. `predict_proba(X)[:,1]` is the probability of **the second class in `classes_`**, not automatically your semantically desired positive label; check `model.classes_`.
- **Multiclass:** exactly one of $K > 2$ classes (types of product). Softmax or tree ensembles can learn directly; One-vs-Rest (OvR) trains K binary models; One-vs-One trains pairwise classifiers. Scoring can use macro, micro or weighted averaging.
- **Multilabel:** each row may have several true labels (e.g., `urgent` and `billing` simultaneously). Target often shape `(n_samples, n_labels)`; `OneVsRestClassifier` with suitable base model is useful. Precision/recall can be aggregated per label, micro/macro, samples-based; subset accuracy requires *all* labels correct and is strict.
- **Ordinal classification:** categories have a meaningful ordering (low/medium/high), but ordinary multiclass metrics ignore distance between misclassification levels; consider ordinal approaches or custom costs.

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.multiclass import OneVsRestClassifier
X3, y3 = load_iris(return_X_y=True)
model = LogisticRegression(max_iter=1000).fit(X3, y3)
print(model.classes_, model.predict_proba(X3[:2]))
# Multilabel: y_multi is an n_samples × n_labels 0/1 matrix;
# OneVsRestClassifier(LogisticRegression(max_iter=1000)).fit(X, y_multi)
```

Imbalance averaging: `macro` unweighted mean over classes (gives minority equal voice); `weighted` weights by each class's support; `micro` aggregates TP/FP/FN globally. For single-label multiclass, micro precision=recall=F1=accuracy when all classes included; not true in general multilabel settings.

32. Classification metrics and threshold selection

Confusion matrix (binary)

Rows = true labels, columns = predicted labels in scikit-learn with explicit label order `[0,1]`:

	Predicted 0	Predicted 1
Actual 0	TN	FP
Actual 1	FN	TP

- **TP:** actual positive, predicted positive.
- **FP:** actual negative, predicted positive (false alarm).
- **FN:** actual positive, predicted negative (missed case).
- **TN:** actual negative, predicted negative.

Worked example: TP=40, FP=10, FN=20, TN=130 (200 samples). Accuracy=$(40+130)/200 = .85$; precision=$40/(40+10) = .80$; recall=$40/(40+20) = .667$; F1=$2(.8)(.667)/(.8+.667) \approx .727$; specificity=$130/(130+10) = .929$. A useful model choice depends on error costs, not on which number sounds largest.

Metric	Equation	Interpretation	Limitation
Accuracy	$(TP+TN)/N$	Fraction all correct	Misleading on imbalanced classes
Precision (PPV)	$TP/(TP+FP)$	Among predicted positives, fraction true	Depends on prevalence
Recall / sensitivity / TPR	$TP/(TP+FN)$	Among actual positives, fraction caught	Can rise by predicting many positives
Specificity / TNR	$TN/(TN+FP)$	Among negatives, fraction correctly rejected	Does not measure missed positives
FPR	$FP/(FP+TN)$	False-alarm rate among negatives	Differs from FDR
FNR	$FN/(FN+TP)$	Miss rate among positives	$1-$ recall
F1	$2PR/(P+R)$	Harmonic precision/recall	Ignores TN and error costs
Balanced accuracy	$(TPR+TNR)/2$	Balances classwise recall (binary)	Still cost-insensitive
MCC	$(TP\cdot TN - FP\cdot FN)/\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}$	Correlation-style balance	Edge cases require care

Metric	Equation	Interpretation	Limitation
ROC-AUC	Area under TPR vs FPR curve	Probability ranking a random positive above random negative (ties handled)	May look optimistic in rare-positive settings
Average precision (AP)	Weighted average of precision at recall jumps	Precision–recall ranking summary	Not identical to trapezoidal PR-AUC
Log loss	$-\frac{1}{n}\sum[y \ln p + (1-y)\ln(1-p)]$	Quality of probability predictions	Penalizes confident wrong probabilities
Brier score	$\frac{1}{n}\sum(p_i - y_i)^2$	Squared probability error	Depends on prevalence, combines calibration/discrimination

Terminology: `ROC-AUC` depends on scores/ranking over many thresholds; `average_precision_score` is **not** exactly area from `auc(recall,precision)` trapezoids; don't call them numerically equivalent. Accuracy and F1 need hard predictions; AUC and proper probability losses need scores/probabilities. **Positive label convention matters.** If a class is absent, some metrics are undefined or given configurable zero-division behavior.

```
from sklearn.metrics import (confusion_matrix, classification_report, accuracy_score,
    precision_score, recall_score, f1_score, balanced_accuracy_score,
    roc_auc_score, average_precision_score, log_loss, brier_score_loss,
    roc_curve, precision_recall_curve)
proba = pipe.predict_proba(X_test)[:,1]     # from earlier LogisticRegression pipeline
threshold = .50
pred = (proba >= threshold).astype(int)
print(confusion_matrix(y_test, pred, labels=[0,1]))
print(classification_report(y_test,pred,zero_division=0))
print('ROC AUC', roc_auc_score(y_test, proba))
print('AP', average_precision_score(y_test, proba))
print('log loss', log_loss(y_test, proba))
print('Brier', brier_score_loss(y_test, proba))
```

Thresholds: precision–recall tradeoff

The classifier may output a probability p, but the decision rule needs a threshold t: predict 1 if $p \geq t$. **Lower threshold** usually raises recall and false positives; **higher threshold** often raises precision but misses more positives. These are typical patterns, not mathematical guarantees for every change or finite sample. Select t using validation predictions / cross-validation, then keep it fixed for the final test.

Example: In an educational fraud-alert simulation, flagging more cases may catch more true events but overwhelm review capacity; choose threshold based on observed validation costs/alert budget. **Correct procedure:** train CV folds → collect out-of-fold scores → select threshold with a cost function or constraint → re-fit model → score untouched test. Do not tune on test. Scikit-learn has threshold-tuning utilities; always follow documented CV rules.

```
from sklearn.metrics import precision_recall_curve
# score_val and y_val must come from a held-out development validation fold
# precision, recall, thresholds = precision_recall_curve(y_val, score_val)
# For a chosen validated threshold t:
# final_pred = (final_model.predict_proba(X_test)[:, 1] >= t).astype(int)
```

Probability calibration

A predicted `0.8` should correspond approximately to an 80% observed positive rate among suitably grouped cases if probabilities are calibrated. **Reliability diagram** plots predicted vs empirical positive rates; `CalibratedClassifierCV` can calibrate SVM, forest and others using proper folds. Calibration and discrimination are different: a model can rank exceptionally while providing poor absolute probabilities. Assess both for decision-making.

```
from sklearn.calibration import CalibratedClassifierCV
from sklearn.svm import LinearSVC
cal = CalibratedClassifierCV(make_pipeline(StandardScaler(), LinearSVC()), cv=3)
cal.fit(X_dev, y_dev)
print(cal.predict_proba(X_test[:3]))
```

Cost matrix and real evaluation

A false positive and a false negative may have very different financial/operational consequences. Define total cost:

$$\text{Cost} = C_{FP} \cdot FP + C_{FN} \cdot FN,$$

possibly with additional costs for investigations, interventions, delayed outcomes, and capacity constraints. **Separate ranking quality, probability calibration and policy/decision threshold.** Always report confusion matrix, prevalence, positive-class meaning, and held-out metrics—not just "accuracy 95%".

Classification model selection card

- Need interpretable baseline + probabilities → **Logistic Regression**, assess calibration.
- Sparse text data → **TF-IDF + Linear SVM / Logistic / Naive Bayes**.
- Small sample, scaled complex margin → **RBF SVM**.
- Explainable threshold decisions → **small/pruned tree**.
- Strong reliable nonlinear tabular baseline → **Random Forest**.
- Maximized tabular predictive signal with controlled tuning → **Boosted Trees**.
- Nearby-instance reasoning, limited features → **KNN**.
- Gaussian class structure → **LDA/QDA**.
- Huge sparse/streaming classification → **SGDClassifier**.

Chapter 32 oral challenge: Class prevalence drops from 20% to 1% while TPR and FPR stay constant. What happens to precision? *It usually falls dramatically: many more negatives are available to create false positives.*

Part V — Unsupervised Learning

Chapters 33–39 · Find structure without a predefined supervised target. Clusters are *hypotheses about similarity*, not automatically real customer personas or truth labels. Scale, distance, feature selection and domain context shape everything.

33. Clustering fundamentals and algorithm choice

Clustering: partition/describe observations according to similarity. Define first what similar means operationally: purchase behavior? spending scale? geography? A distance-based algorithm may group by large-scale features rather than relevant business patterns if preprocessing is careless.

Requirements checklist: Is n huge? Are clusters roughly spherical and equally sized? Are there outliers? Are clusters expected to be nested, elongated, different densities? Must every row belong to a group? Must future rows receive a cluster quickly? Is cluster count known?

Algorithm	Assumed/typical shapes	Need k?	Noise label?	New-row `predict()`?
K-Means	Roughly compact/spherical, similar scale	Yes	No	Yes
DBSCAN	Density-connected, arbitrary shape	No	Yes (`-1`)	Not in standard sklearn DBSCAN

Algorithm	Assumed/typical shapes	Need k?	Noise label?	New-row `predict()`?
Agglomerative	Hierarchical; depends on linkage/distance	Usually `n_clusters` or threshold	No native noise	Not typically
Gaussian Mixture	Soft elliptical components	Yes	Not inherent	Yes, with probabilities
HDBSCAN (optional)	Variable density hierarchy	No fixed k	Yes	Depends on implementation

General clustering workflow: remove identifiers/leakage, handle missing data and scaling, choose similarity metric, fit only on development data if clusters will feed supervised prediction, evaluate multiple seeds/stability/plots, profile clusters using original units, and validate whether distinctions correspond to useful outcomes. Silhouette score alone does not prove business usefulness.

34. K-Means and K-Means++

Idea: Choose k centroids $\mu_1, \dots, \mu_k$, assign each row to nearest centroid, update centroids to mean of assigned rows, repeat until convergence. It seeks low **within-cluster sum of squares** (inertia):

$$J = \sum_{i=1}^{n} \|x_i - \mu_{c_i}\|_2^2.$$

The objective is nonconvex; different initializations may yield different local optima. **K-Means++** seeds centroids far apart probabilistically to improve starts compared with naive random seeding; it does not guarantee global optimum. `n_init` repeats initialization and picks best inertia; set explicit value for reproducibility if desired.

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
X, _ = make_blobs(n_samples=500, centers=4, cluster_std=.8, random_state=42)
X_scaled = StandardScaler().fit_transform(X)  # unsupervised descriptive demo
model = KMeans(n_clusters=4, init='k-means++', n_init=10, random_state=42)
cluster = model.fit_predict(X_scaled)
print('inertia:', model.inertia_)
print('centroids (scaled):', model.cluster_centers_)
```

Hyperparameters: `n_clusters`, `init`, `n_init`, `max_iter`, `tol`. **Limitations:** sensitive to scale, outliers, initializations, elongated/nonconvex clusters, and uneven densities. **Soute-nance:** What is inertia? *Sum of squared distances from points to assigned centroids; always tends to decrease as k increases, so it cannot alone choose the best k.*

35. DBSCAN and hierarchical clustering

DBSCAN

Density-Based Spatial Clustering of Applications with Noise groups points connected through dense neighborhoods. A **core** point has at least `min_samples` points within radius eps (including itself in scikit-learn). **Border** points are reachable from a core but may lack dense neighborhoods themselves; **noise** points get label `-1`. DBSCAN can discover nonconvex shapes without selecting k.

```
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons
X_moons, _ = make_moons(n_samples=400, noise=.07, random_state=42)
X_moons = StandardScaler().fit_transform(X_moons)
labels = DBSCAN(eps=.25, min_samples=5).fit_predict(X_moons)
print('labels, including -1 noise:', set(labels))
```

Hyperparameters: `eps` (distance threshold), `min_samples`, `metric`. Small eps may mark most points noise; large eps may merge distinct groups. Sensitive to varying densities and high di-

mensions; expensive neighborhood search in some conditions. **Do not treat -1 as an ordinary "noise cluster" in silhouette without deciding evaluation protocol.** The sklearn DBSCAN object has no standard `.predict()` for new observations.

Hierarchical / agglomerative

Start with one cluster per observation, iteratively merge nearest pairs of clusters. A **dendrogram** records merge distance and reveals nested partitions. Linkage choices: `single` (closest pair; chain risk), `complete` (farthest pair; compact), `average` (average intercluster distance), `ward` (minimum variance with Euclidean geometry).

```
from sklearn.cluster import AgglomerativeClustering
hier = AgglomerativeClustering(n_clusters=2, linkage='ward')
labels = hier.fit_predict(X_moons)
print('cluster counts:', np.unique(labels, return_counts=True))
```

Soutenance: DBSCAN vs K-Means? *DBSCAN learns density-connected regions and can flag noise; K-Means assumes every row belongs to one of k compact centroid groups.*

36. Gaussian Mixture Models (GMM)

Idea: Assume samples arise from a weighted mixture of K Gaussian distributions. Instead of hard nearest-centroid memberships, estimate soft membership probabilities:

$$p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k), \qquad \sum_k \pi_k = 1.$$

EM algorithm: E-step computes responsibilities $P(z = k \mid x)$ using current parameters; M-step updates mixture weights, means and covariance matrices to maximize likelihood (locally). Unlike K-Means, covariance allows elliptical components, soft assignments and likelihood-based comparisons.

```
from sklearn.mixture import GaussianMixture
gmm = GaussianMixture(n_components=4, covariance_type='full', n_init=5, random_state=42)
gmm.fit(X_scaled)
print(gmm.predict(X_scaled[:3]))
print(gmm.predict_proba(X_scaled[:3]))
print('BIC:', gmm.bic(X_scaled), 'AIC:', gmm.aic(X_scaled))
```

Model selection: AIC/BIC penalize mixture complexity under modeling assumptions; lower typically preferred *for comparable fitted mixtures/data*. **Hyperparameters:** number of components, covariance type (`full`,`diag`,`spherical`,`tied`), `reg_covar` for numerical stability, `n_init`. **Limitations:** Gaussian assumptions, potential singular covariance, sensitivity to seeds, likelihood may not represent business segmentation. **Oral:** Why is GMM "soft" clustering? *Each row receives a probability of membership in each component.*

37. Evaluating clustering

Internal evaluation measures geometry without ground-truth labels. **External evaluation** compares to genuine known classes (if available) but those labels may not match clustering's purpose.

Metric	What it measures	Preference	Important warning
Inertia / WCSS	Sum squared point-to-centroid distance	Lower for fixed k context	Monotonically drops as k increases
Elbow	Visual slope change of inertia vs k	Distinct bend	Subjective; often no clear elbow
Silhouette	$(b-a)/\max(a,b)$ where a within-cluster mean distance, b nearest-other-cluster mean	Higher, max 1	Defined for 2 to $n-1$ distinct labels; favors separated/compact groups

Metric	What it measures	Preference	Important warning
Davies–Bouldin	Average similarity to most similar competing cluster	Lower	Depends on geometry, scale
Calinski–Harabasz	Between-cluster vs within-cluster dispersion ratio	Higher	Can favor particular k/shapes
ARI	Adjusted Rand Index vs external true labels	Higher, max 1	Requires external labels
NMI	Normalized mutual information with labels	Higher	Requires external labels
Stability	Similar clusters across resamples/seeds	More stable	Matching clusters across runs requires care

Silhouette worked example: Point has average intra-cluster distance $a = 2$, nearest other-cluster average distance $b = 5$; $s = (5-2)/5 = .6$ → relatively well separated. If $a > b$, silhouette is negative → point may fit another cluster better.

```
from sklearn.metrics import (silhouette_score, davies_bouldin_score,
                             calinski_harabasz_score)
for k in range(2, 8):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X_scaled)
    labs = km.labels_
    print(k, round(km.inertia_, 1),
          round(silhouette_score(X_scaled, labs), 3),
          round(davies_bouldin_score(X_scaled, labs), 3),
          round(calinski_harabasz_score(X_scaled, labs), 1))
```

Warning: This code is descriptive analysis. If cluster labels/centroids become features for supervised classification, fit the clusterer *within training folds* of that supervised pipeline, not across all rows. A cluster with a number 2 is not necessarily "better" than cluster 1: labels are arbitrary and not ordinal.

38. Dimensionality reduction: PCA, t-SNE, UMAP and SVD

PCA (Principal Component Analysis)

Goal: Rotate centered numeric data to new orthogonal axes (principal components) ordered by explained variance. First component is the direction of highest variance; next captures highest remaining variance subject to orthogonality. For centered X, covariance $S = X^\top X/(n-1)$; component directions are its eigenvectors (implemented efficiently using SVD). PCA is **not target-aware** and high variance does not always imply predictive usefulness.

```
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline
pca_pipeline = make_pipeline(StandardScaler(), PCA(n_components=2))
Z = pca_pipeline.fit_transform(X_scaled)  # illustration: data already scaled
print(Z.shape)
print(pca_pipeline[-1].explained_variance_ratio_)
```

Hyperparameters: `n_components` integer or explained-variance fraction (depending on solver support), `whiten`, SVD solver. **Interpret:** loadings explain direction composition, explained-variance ratio shows variance kept, reconstructed error shows lost information. Scaling before PCA helps prevent wide-units features from dominating; if raw variance is semantically meaningful, scaling may be inappropriate. Fit inside CV when PCA is part of a supervised model. A PCA scatter plot is a projection, not proof of true clusters.

TruncatedSVD

For sparse high-dimensional matrices (e.g., TF-IDF), `TruncatedSVD` provides low-rank approximation **without centering** (centering would often destroy sparsity). Components need not behave like centered PCA; interpret explained variance accordingly.

t-SNE

A nonlinear **visualization** technique preserving local neighbor relationships in a low-dimensional embedding. Distances between far-apart groups, group sizes and absolute axes are **not reliably interpretable**. `perplexity`, `learning_rate`, initialization, sample size and seed change appearances; do not infer cluster validity from pretty islands.

```
from sklearn.manifold import TSNE
Z_vis = TSNE(n_components=2, perplexity=30, random_state=42).fit_transform(X_scaled)
print(Z_vis.shape)  # visualization only, fit on appropriate development data
```

UMAP

A manifold-based low-dimensional visualization/representation library (optional `umap-learn`). `n_neighbors` balances local/global structure, `min_dist` controls low-dimensional compactness, `metric` defines similarity. Different settings change pictures; if used as model input, fit within a split-safe pipeline, not on all data before evaluation.

```
# pip install umap-learn
import umap
reducer = umap.UMAP(n_neighbors=15, min_dist=.1, n_components=2, random_state=42)
Z_umap = reducer.fit_transform(X_scaled)
```

Soutenance compare: PCA is linear, oriented around global variance and transformable; t-SNE emphasizes local visualization, and UMAP emphasizes manifold neighbor structure with tunable locality. Do not describe t-SNE or UMAP coordinates as stable physical measurements.

39. Anomaly detection: Isolation Forest, LOF, One-Class SVM

What is an anomaly? A case rare/atypical relative to the training distribution—not automatically a defect, threat or error. Choose whether the problem is **outlier detection** (fit with mostly typical data, perhaps contaminated) or **novelty detection** (fit on presumed normal data, then score genuinely new observations). Carefully specify base rates and false alarm costs.

Model	Mechanism	Main controls	Pros / limits
Isolation Forest	Unusual points isolated in fewer random partitions	`n_estimators`, `max_samples`, `contamination`	Scales well; randomness; interpretability limited
Local Outlier Factor (LOF)	Compare local density with neighbor density	`n_neighbors`, `contamination`, `metric`	Finds local density anomalies; standard LOF lacks out-of-sample `predict` unless `novelty=True`
One-Class SVM	Boundary enclosing typical input region	`nu`, `gamma`, `kernel`	Flexible, scale-sensitive, costly for large n

```
from sklearn.pipeline import make_pipeline
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.svm import OneClassSVM
iso = IsolationForest(contamination=.05, random_state=42).fit(X_scaled)
print('IF labels:', np.unique(iso.predict(X_scaled), return_counts=True))  # -1 anomaly
lof = LocalOutlierFactor(n_neighbors=20, contamination=.05)
print('LOF labels:', np.unique(lof.fit_predict(X_scaled), return_counts=True))
oc = make_pipeline(StandardScaler(), OneClassSVM(nu=.05, gamma='scale'))
oc.fit(X_scaled)
print('OCSVM labels:', oc.predict(X_scaled[:5]))
```

`contamination` often sets an output threshold/expected fraction, not “truth that 5% are fraud”. Distinguish `score_samples`, `decision_function`, `predict`, and which direction means more anomalous across implementations. **Evaluate with labels if available** using recall/precision

at review capacity; otherwise manual inspection, stability, downstream utility. Hold out temporal groups to test actual novelty detection.

Clustering segmentation project: recommended report

1. Define behavioral features and business goal, omit identifiers.
2. Inspect and transform skewed measures; compare scaling choices.
3. Try k=2…10, plot inertia/silhouette and cluster size.
4. Try DBSCAN for noisy/nonconvex patterns; document eps/min_samples.
5. Profile clusters in **original units** with median/spend/churn per group (analysis only).
6. Test stability across seeds/resamples and distribution shifts.
7. Report why segments are actionable, limitations, and how new customers will be assigned.

Oral challenge: Why might K-Means with the highest silhouette score not be the best production segmenter? *It can group by meaningless dimensions, create unusably small segments, or yield clusters with no operational action.*

Part VI — Professional ML Workflows

Chapters 40–47 · Transform a notebook model into a reproducible, validated, deliverable solution. Most mistakes in real ML projects occur outside the algorithm itself: ambiguous target, leakage, bad split, wrong metric, unsupported schema, poor monitoring.

40. Scikit-learn Pipeline, ColumnTransformer and reusable preprocessing

Why pipelines matter

A `Pipeline` composes transformations and a final estimator in the order they are applied. On `fit`, each transformer learns only from the training subset; on `predict`, previously learned transformations apply without recomputing from test data. Within cross-validation, the entire pipeline is re-fit independently for each fold, which prevents many common leakage errors.

Transformers: implement `.fit()` (learn state) and `.transform()` (apply state), optionally `.fit_transform()`. **Estimators:** `.fit()` plus `.predict()` or `.predict_proba()` where applicable. **ColumnTransformer:** routes different feature columns through different preprocessing branches, concatenates outputs, and integrates with Pipeline.

Complete mixed-data classification pipeline:

```
import pandas as pd
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, classification_report

# Replace with your actual data, preserving the observation unit/target date.
# df = pd.read_csv('customers.csv')
# target = 'churn'
# X, y = df.drop(columns=[target]), df[target]
# numeric_features = ['age', 'monthly_charge', 'tenure']
# categorical_features = ['contract', 'payment_method']

num = Pipeline([
    ('impute', SimpleImputer(strategy='median', add_indicator=True)),
    ('scale', StandardScaler())
])
cat = Pipeline([
    ('impute', SimpleImputer(strategy='most_frequent')),
    ('encode', OneHotEncoder(handle_unknown='ignore'))
])
preprocess = ColumnTransformer([
    ('numeric', num, numeric_features),
    ('categorical', cat, categorical_features),
], remainder='drop')
model = Pipeline([
```

```
    ('preprocess', preprocess),
    ('classifier', LogisticRegression(max_iter=2000, class_weight='balanced'))
])
X_dev, X_test, y_dev, y_test = train_test_split(
    X, y, test_size=.2, stratify=y, random_state=42)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_validate(model, X_dev, y_dev, cv=cv,
                           scoring={'f1':'f1', 'ap':'average_precision'},
                           return_train_score=True)
print('CV AP:', cv_scores['test_ap'].mean())
model.fit(X_dev, y_dev)
print(classification_report(y_test, model.predict(X_test)))
print('Final AP:', average_precision_score(y_test,
                                           model.predict_proba(X_test)[:,1]))
```

The above is a **template**: define X, y, numeric/categorical feature lists on your dataset before running. For a fully runnable example with synthetic data, see `code/classification_pipeline.py`.

Critical detail: `add_indicator=True` can alter output column count depending on columns missing during fit; ensure schema is fixed. Categorical types should be converted consistently, and production columns must match training. `OneHotEncoder(handle_unknown='ignore')` maps unseen categories to all zeros for that feature; it does not solve category drift. **Feature names:** `model.named_steps['preprocess'].get_feature_names_out()` helps interpret transformed features where supported.

Creating a custom transformer

Keep transformations deterministic, stateful learned parameters in `.fit()` only, and test array/DataFrame behavior. Use `FunctionTransformer` for simple stateless operations; for trainable custom transformations implement `BaseEstimator`, `TransformerMixin`. A transformer that computes statistics from the complete dataset in advance defeats the pipeline's leakage guarantees.

Resampling pipelines

`imblearn.pipeline.Pipeline` integrates SMOTE and other samplers to occur only during training/CV, not at prediction. Ordinary scikit-learn `Pipeline` is for `.transform()` components and an estimator, not SMOTE's `fit_resample` contract.

41. Cross-validation, GridSearchCV, RandomizedSearchCV and tuning

Cross-validation mental model

K-fold: Split development data into K folds. Fit on $K-1$, validate on remaining, repeat, aggregate metric mean and spread. It gives more information than one split but folds are not fully independent, so standard deviation across folds is not automatically a confidence interval.

- **KFold:** generic IID data; shuffle optionally.
- **StratifiedKFold:** approximate class proportions for supervised classification.
- **GroupKFold / StratifiedGroupKFold:** keep repeated entities from straddling folds.
- **TimeSeriesSplit:** train on past, validate on later time ranges; choose adequate gap, forecast horizon and label availability.
- **RepeatedStratifiedKFold:** repeated randomized folds improve sense of split variability; avoid for grouped/time data without adaptation.
- **Nested CV:** outer folds estimate full tuning workflow; inner folds select hyperparameters. Useful when selecting many models on small datasets.

Grid and randomized search

GridSearchCV enumerates specified combinations; `RandomizedSearchCV` samples a fixed number of combinations from distributions/lists (often more efficient with many hyperparameters). **Hyperparameters** are selected from validation/CV, not the test set. `refit=True` retrains selected pipeline on full development data. `best_score_` is the selected CV score and is **optimistically selected** if you treat it as an independent final measurement.

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
base = make_pipeline(StandardScaler(), SVC())
search = GridSearchCV(
    estimator=base,
    param_grid={'svc__C':[.1, 1, 10], 'svc__gamma':['scale', .01, .1],
                'svc__kernel':['rbf']},
    cv=StratifiedKFold(5, shuffle=True, random_state=42),
    scoring='balanced_accuracy', n_jobs=-1, refit=True)
# search.fit(X_dev, y_dev)
# print(search.best_params_, search.best_score_)
# y_test_pred = search.best_estimator_.predict(X_test)  # final audit
```

Random search example:

```
from scipy.stats import loguniform
from sklearn.model_selection import RandomizedSearchCV
search = RandomizedSearchCV(
    base, param_distributions={'svc__C':loguniform(.01, 100),
                               'svc__gamma':loguniform(.0001, 1)},
    n_iter=25, scoring='roc_auc', cv=5, random_state=42, n_jobs=-1)
# search.fit(X_dev, y_dev)
```

Bayesian optimization / Optuna: Adaptive samplers choose new trials based on earlier validation results; useful for expensive tuning, but still requires leak-free CV and a truly untouched test. **Early stopping:** internal validation must not be the final audit data. **Compute budgeting:** estimate number of fits as $\#\text{candidates} \times K$ plus refit; use simpler models and smaller searches as baselines.

Soutenance: Why is nested CV sometimes preferable? *Non-nested CV uses the same score to choose the best model and report its quality, creating selection bias; an outer loop measures the whole tuning procedure.*

42. Model comparison and baselines

Best model is conditional on context. Consider predictive quality, inference time, training cost, interpretability, fairness, calibration, drift robustness, maintenance complexity and deployment constraints.

1. Define the unit, target, positive class and prediction time.
2. Select one primary evaluation metric consistent with real-world costs.
3. Build dummy baseline; build interpretable simple baseline.
4. Use same development folds, same features/safe preprocessing protocol for each candidate.
5. Compare variability, confusion matrix / residuals and subgroup errors.
6. Tune a limited number of promising models; record resources and settings.
7. Lock model/threshold; report final holdout metrics once.

Avoid winner's curse: Testing many configurations and then reporting only the highest CV score without uncertainty exaggerates performance. If the model selection changes after final test inspection, that test is no longer untouched.

Example comparison template:

```
from sklearn.model_selection import cross_validate
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
models = {
  'dummy': DummyClassifier(strategy='prior'),
  'logistic': make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)),
  'forest': RandomForestClassifier(n_estimators=200, random_state=42),
}
for name, model in models.items():
    cv = cross_validate(model, X_dev, y_dev, cv=5,
        scoring=['balanced_accuracy','average_precision'])
    print(name, {k:round(v.mean(), 3) for k,v in cv.items() if k.startswith('test_')})
```

Business baseline vs statistical baseline: An existing rule-based workflow may be more operationally relevant than `DummyClassifier`. Compare incremental gains and deployment cost.

43. Interpreting models: coefficients, importance, permutation, SHAP

Model-specific / intrinsic interpretation: Standardized linear coefficients indicate direction/magnitude under assumptions; trees show thresholds; `feature_importances_` typically summarizes impurity decreases/booster-defined importance, not causality. Correlated features split importance across alternatives; models can use proxies for sensitive traits without including protected attributes directly.

Permutation importance: Measure drop in held-out metric after shuffling one feature's values, holding model fixed. Good for model reliance, but correlated features can mask each other's importance; permutations may create unrealistic combinations. Work with appropriate held-out/CV rows. **Metric selection matters:** Use `scoring='neg_mean_absolute_error'` for regression MAE context and `roc_auc` / `average_precision` / F1 as justified.

```
from sklearn.inspection import permutation_importance
# model is already fit on development data; X_test, y_test remain untouched until final audit.
# For development interpretation, use a held-out validation portion instead of final test.
# result = permutation_importance(model, X_val, y_val, n_repeats=15,
#                                 random_state=42, scoring='roc_auc')
# sorted(zip(X_val.columns, result.importances_mean), key=lambda x:-x[1])
```

Partial dependence plot (PDP): average model prediction over altered values of a feature, potentially extrapolating into unrealistic combinations when features are correlated. **ICE:** individual predictions as one feature changes. **SHAP:** Shapley-value-based contribution allocation under a chosen background/reference and explanation model. SHAP values explain a **particular predictive model's output** relative to a background; they are **not causal impacts**. Compare log-odds vs probability output space and model-specific/approximation restrictions.

```
# Optional: pip install shap
# import shap
# explainer = shap.Explainer(trained_tree_model, X_dev_sample)
# values = explainer(X_val_sample)
# shap.plots.beeswarm(values)
```

Soutenance: Are features with high importance causes of churn? *No. Importance describes a model's predictive reliance given correlated data, selection and measurement biases.*

44. Save and load a complete model with Joblib

Save the **entire fitted pipeline** (imputation, encoding, scaling, model), not just the final estimator. Otherwise inference may use different transformations. Version package environment and record schema/training metadata. `joblib`/pickle formats can execute code when loading: **never load untrusted model files** and do not assume arbitrary old files work across dependency versions.

```
from pathlib import Path
import joblib
Path('models').mkdir(exist_ok=True)
# pipeline.fit(X_dev, y_dev)  # complete pipeline from ch. 40
# joblib.dump(pipeline, 'models/churn_pipeline.joblib')
# loaded = joblib.load('models/churn_pipeline.joblib')  # trusted artifacts only
# loaded.predict(new_rows[training_feature_names])
```

Store alongside: model version, Python and library versions, training dataset snapshot/hash and dates, feature schema/order/types, target semantics, positive class, chosen threshold and metric, CV scores, limitations, ownership, retraining criteria. For library-varying persistence and trusted alternatives see current sklearn `model_persistence` docs.

45. MLflow experiment tracking

Purpose: Record experiments, their parameters, metrics, tags and artifacts so you can explain *which model was trained, on what data, with what settings and why it won.* An experiment contains many runs; each run is one tracked execution. The tracking server/UI is different from model registry and deployment.

```
# Optional: pip install mlflow
import mlflow
import mlflow.sklearn
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
X, y = load_iris(return_X_y=True)
X_train, X_val, y_train, y_val = train_test_split(
    X, y, stratify=y, test_size=.25, random_state=42)
params = {'max_iter':1000, 'random_state':42}
model = LogisticRegression(**params).fit(X_train, y_train)
acc = accuracy_score(y_val, model.predict(X_val))
mlflow.set_experiment('classical_ml_learning')
with mlflow.start_run(run_name='logistic_iris_baseline'):
    mlflow.log_params(params)
    mlflow.log_metric('validation_accuracy', acc)
    mlflow.set_tag('dataset', 'sklearn_iris')
    mlflow.sklearn.log_model(sk_model=model, name='iris_logistic')
```

Launch UI from PowerShell: `mlflow server --port 5000`, then visit `http://127.0.0.1:5000`. For external tracking servers configure `mlflow.set_tracking_uri('http://...')` and authentication securely. Official current MLflow docs use `name=` when logging sklearn models; older snippets sometimes use `artifact_path=`. Do not expose sensitive data in tracked artifacts.

Good experiment name: `churn_prediction_2026Q4`; log training data hash, model family, chosen scoring, fold scheme, random seed, feature list version, holdout/validation distinction. **Registry:** optional approval/versioning workflows; a "champion" alias points to a selected registered model version, but selection must still be validated independently.

46. Model deployment: FastAPI and Streamlit

Deployment isn't just serving `predict()`. Your inference interface must validate input schema, use the same feature order/types, preserve categories, avoid leaking model source, enforce access controls, monitor latency and errors, and decide what predictions mean to users.

FastAPI minimal illustration:

```
# In code/api_example.py, we provide a runnable self-contained example.
from fastapi import FastAPI
from pydantic import BaseModel
from sklearn.linear_model import LogisticRegression
import numpy as np
app = FastAPI()
# In a real project, load a versioned trusted fitted pipeline, not a dummy trained model.
_demo = LogisticRegression().fit([[0.,0.],[1.,1.],[0.,1.],[1.,0.]], [0,1,0,1])
class Features(BaseModel):
    age: float
    monthly_charge: float
@app.post('/predict')
def predict(payload: Features):
    x = np.array([[payload.age, payload.monthly_charge]], dtype=float)
    return {'class':int(_demo.predict(x)[0])}
```

This is an **API mechanics demo only**; a meaningful classifier must use a real trained pipeline and validated feature schema, with appropriate positive-class interpretation. Start with `uvicorn code.api_example:app --reload` after installing dependencies, then inspect `/docs` locally. Do not serve development reload publicly.

Streamlit presentation:

```
# Conceptual UI: a complete example is in code/streamlit_example.py
import streamlit as st
st.title('Classical ML demo')
age = st.number_input('Age', min_value=0, max_value=120, value=30)
charge = st.number_input('Monthly charge', min_value=0.0, value=80.0)
if st.button('Predict'):
    st.write('Inputs:', age, charge)
    st.info('Attach a validated and versioned pipeline to make predictions.')
```

Serving checklist: define prediction contract; provide model version; document unsupported ranges; use HTTPS/auth where appropriate; do not reveal training rows; handle invalid/null/categorical values; log non-sensitive metadata; implement rollback and human oversight for consequential decisions.

47. Monitoring, drift, reproducibility and responsible ML

Drift types:

- **Covariate/data drift** $P(X)$ changes: customer ages/prices change.
- **Label/prior shift** $P(Y)$ changes: churn prevalence changes.
- **Concept drift** $P(Y \mid X)$ changes: relationship between predictors and target changes.
- **Schema/quality drift:** columns, units, missing values, category dictionary change.

Monitoring signals: incoming distributions vs reference (quantile charts, PSI with cautious interpretation, statistical tests), missingness, invalid values, unseen categories, inference latency/errors, model confidence, prediction rate, realized outcome metrics after delayed labels, subgroup disparities, complaints/human review. A detected statistical distribution difference does *not* automatically require retraining; diagnose business and measurement causes.

Reproducibility checklist: freeze dependencies, record dataset version/hash and schema, track feature definitions and target horizon, fix seeds while acknowledging nondeterminism, version pipelines, log splits and CV strategy, use tests/data validation, store model card and limitations. Beware data training/serving skew (different encoders or missingness policy between notebook and API).

Responsible ML: consent/legal basis; minimize personal data; avoid unjustified proxies; test subgroup reliability where appropriate and lawful; restrict access to sensitive artifacts; consider explainability and human appeals for consequential predictions. Avoid unsupported medical/financial claims. Monitor differences in harm or false positives across relevant groups in addition to headline accuracy.

Fairness and metrics: Equality of statistical metrics across groups may conflict when base rates differ; there is no single universally correct fairness definition. Choose criteria based on legal/domain context with stakeholders; document tradeoffs and uncertainty.

Full professional pipeline checklist (before viva or deployment)

- ☐ Target definition and future prediction moment clear.
- ☐ Dataset rows, timestamps, groups, units and provenance documented.
- ☐ Right split; final test untouched; imbalanced prevalence known.
- ☐ All data-learned transformations inside CV pipeline; no future proxies.
- ☐ Dummy and interpretable baselines measured.
- ☐ Primary metric and tradeoff metric justified.
- ☐ CV and hyperparameter search reproducible and appropriate.
- ☐ Error analysis (residuals/confusion matrix, time and groups) complete.
- ☐ Probabilities calibrated/threshold tuned on development data if necessary.
- ☐ Test evaluation performed once after choices locked.
- ☐ Full pipeline, schema, environment and model card versioned.
- ☐ API/UI validated, monitoring and rollback plan documented.

Soutenance summary: “The value of my pipeline isn’t only the model score; it is that the reported score was earned without information leakage, the data match the future decision, and the final artifact reproduces preprocessing exactly.”

Part VII — Soutenance Croisée Preparation

Chapters 48–51 · Defend your decisions, not just your vocabulary. Answer every oral question in the pattern: **definition → intuition → technical detail → why it mattered in your project → limitation**.

48. Model comparison — which algorithm, and why?

Regression comparison

Model	Nonlinear?	Must scale?	Handles outliers?	Interpretation	Common risk
Linear / Multiple Linear	Linear in features	Not for raw OLS fit, but helpful for regularization	Sensitive	Coefficients	Misses nonlinear effect
Polynomial + Ridge	Yes in original x	Recommended	Sensitive	Feature coefficients complex	High-degree overfit
Ridge	Linear	Strongly recommended	Moderate sensitivity	Coefficients	Underfit nonlinear data
Lasso / ElasticNet	Linear	Strongly recommended	Moderate sensitivity	Sparse/shrunk coefficients	Correlated selection instability
KNN Regressor	Local nonlinear	**Yes**	Local noise sensitivity	Example neighbors	High-dimensional distances
Decision Tree	Piecewise constant	Usually no	Sensitive to noisy leaves	Rules	Variance/overfitting
Random Forest / Extra Trees	Nonlinear	Usually no	Robust averaging, not guaranteed	Importance/partial dependence	Large model, no extrapolation
SVR RBF	Nonlinear	**Yes**	Epsilon loss can be useful	Limited	Quadratic-ish computational burden
Gradient-boosted trees	Nonlinear	Usually no for numeric splits	Loss-dependent	SHAP/importance	Excessive tuning/overfit

Classification comparison

Model	Strength	Weakness	Typical use	Important hyperparameters
Logistic Regression	Interpretable linear log-odds; good baseline	Nonlinear boundary requires features	Churn / tabular	C, penalty, class weight
KNN	Simple local boundaries	Slow inference; scaling	Small datasets	k, distance, weights
Gaussian NB	Very fast, small data	Class-conditional independence	Numeric baseline	var smoothing
Multinomial/Bernoulli NB	Sparse text	Poorly calibrated estimates	Text classification	smoothing alpha
Decision Tree	Clear if/then rules	High variance	Explainable small rules	max depth/leaf
RF / Extra Trees	Strong broad tabular baseline	Less interpretable, potential calibration	Structured mixed data	trees, leaf size, max features
Linear SVM	High-dimensional sparse separator	No direct calibrated probability	Text	C
RBF SVM	Flexible margin	Slow large n, scaling	Medium sized nonlinear	C, gamma
LDA / QDA	Fast generative Gaussian models	Assumption-sensitive	Small, structured continuous	shrinkage/regularization

Model	Strength	Weakness	Typical use	Important hyperparameters
Boosted trees	Strong structured predictive fit	Tuning, drift, opacity	Tabular ML competitions/business	learning rate, depth, iterations
SGDClassifier	Streaming/sparse scale	Convergence and tuning	Very large sparse datasets	loss, alpha, learning rate

The correct model choice explanation

> "I used logistic regression as an interpretable baseline. Random Forest captured nonlinear interactions without requiring numeric scaling. XGBoost improved cross-validated PR-AUC, but I also checked calibration, minority recall, inference cost and stability. I selected the model and threshold on development folds, then reported one untouched test evaluation. Its use is conditional on future data matching the validation design."

Avoid claiming: "I used XGBoost because it is always the best," "I scaled trees because every model needs it," "F1 is always best for imbalance," "High AUC means perfect probabilities," "PCA creates meaningful clusters."

49. Oral technical questions — short, precise answers

Pipeline and data

1. **What is a feature?** A measurable input available at prediction time, e.g. customer tenure.
2. **What is a target?** Output supervised learning tries to predict.
3. **Supervised vs unsupervised?** Supervised learns from labeled inputs; unsupervised seeks structure without task labels.
4. **What is a hyperparameter?** A training-control chosen by model selection (depth, k, C), not learned like tree split thresholds/coefficient values.
5. **Why train/test split?** To estimate performance on held-out cases, not ones that helped learn rules.
6. **Why validation besides test?** Tune models and thresholds without spending the final independent estimate.
7. **What is cross-validation?** Repeated train/validation folds on development data; metric mean/spread estimates selection behavior.
8. **Why stratify?** Preserve approximate class ratios; not enough to prevent same-customer overlap.
9. **What is GroupKFold?** Groups never cross training/validation partitions within a fold.
10. **Why random split invalid for forecasting?** It allows future observations to inform predictions of earlier periods.
11. **What is data leakage?** Future/held-out/target-derived information influences model fitting or selection.
12. **Why use Pipeline?** Ensure identical preprocessing at train/inference and fold-local fitting in CV.
13. **Why fit scaler only on train?** Test mean/variance is unseen information; fitting on all data contaminates evaluation.
14. **What is one-hot encoding?** Represent unordered categories with indicator columns.
15. **What is standardization?** Recenter/rescale using training mean and std: $(x - \mu)/\sigma$.
16. **StandardScaler vs MinMaxScaler?** Standard deviation units vs min–max range; both fit on train only.
17. **How do you handle missing values?** Investigate mechanism, use train-fitted imputation/indicator and validate it.
18. **Why is global SMOTE dangerous?** Synthetic samples can encode validation/test neighbors and leak into training.
19. **What is concept drift?** $P(Y \mid X)$ changes after deployment.
20. **What is class imbalance?** Label prevalence is strongly unequal; accuracy may be misleading.

Regression

21. **What does linear regression learn?** Coefficients minimizing squared residuals, conditional on linear feature representation.
22. **What is a residual?** Observed target minus fitted prediction $y - \hat{y}$.
23. **Difference simple/multiple?** One predictor vs several predictors.
24. **Is polynomial regression nonlinear?** Nonlinear in original inputs; linear in coefficients over polynomial features.
25. **Ridge vs Lasso?** Ridge L2 shrinks all; Lasso L1 may set some exactly zero.
26. **Elastic Net?** Mixes L1/L2, often useful with correlated predictors.
27. **What is R^2?** Proportional reduction in squared error vs predicting the evaluation mean; can be negative.
28. **Why choose MAE vs RMSE?** MAE weights errors linearly; RMSE weights large misses more heavily.
29. **Why MAPE problematic?** Zero/near-zero true values cause undefined or unstable percentage errors.
30. **What is SVR epsilon?** Error tube of tolerated deviations before penalty.
31. **Why might KNN regression fail to extrapolate?** Predictions use weighted averages of observed neighbor targets.

Classification

32. **Why logistic regression is classification?** Sigmoid/softmax turns linear scores into estimated class probabilities.
33. **What is sigmoid?** $1/(1+e^{-z})$, maps real scores to (0,1).
34. **What is a logit?** Log-odds $\log(p/(1-p))$ or unnormalized model score in some contexts.
35. **Precision vs recall?** Precision is correct among predicted positives; recall is caught among actual positives.
36. **Specificity?** Correct rejection rate among actual negatives $TN/(TN+FP)$.
37. **F1?** Harmonic mean of precision and recall; ignores TN.
38. **What is ROC-AUC?** A threshold-independent ranking summary using TPR vs FPR.
39. **Why PR-AUC for rare positives?** Focuses precision and recall of positive class; AP and trapezoid PR-AUC are different implementations.
40. **What is a confusion matrix?** Counts TP, FP, FN, TN (given positive class definition).
41. **Why change classification threshold?** Align predicted actions with FP/FN costs and recall/precision constraints.
42. **What is calibration?** Among predictions near p, the event occurs near fraction p.
43. **What is a support vector?** An influential training point defining an SVM margin boundary.
44. **KNN vs K-Means?** KNN supervised local voting; K-Means unsupervised centroid grouping.
45. **Bagging vs boosting?** Bagging averages randomized learners; boosting trains sequential corrections.
46. **Random Forest vs decision tree?** Forest averages diverse trees, generally lowering variance at added cost.
47. **XGBoost vs Random Forest?** Boosted additive loss minimization vs bagging of randomized trees.
48. **Why not use accuracy alone?** With 1% positives, predicting all negative is 99% accurate and misses all positives.
49. **Naive Bayes independence assumption?** Features conditionally independent given class, often imperfect but useful.
50. **When use QDA not LDA?** Enough data and differing class covariances justify quadratic boundary.

Unsupervised and production

51. **What does K-Means minimize?** Within-cluster sum of squared Euclidean distances to centroids (inertia).
52. **What is K-Means++?** Probabilistic centroid initialization to improve starts.
53. **What is DBSCAN eps?** Radius of local neighborhoods used in density connectivity.
54. **Why label -1 in DBSCAN?** Points classified as noise by the density criterion.
55. **Silhouette score?** Compare within-cluster cohesion a and next-cluster separation b.

56. **PCA vs t-SNE?** PCA linear global variance directions; t-SNE nonlinear local-neighborhood visualization.
57. **Why scale before PCA?** Avoid arbitrary feature units dominating total variance, when appropriate.
58. **Isolation Forest intuition?** Unusual points need fewer random splits to isolate.
59. **Why save full pipeline?** Reproducible transforms and model predictions with consistent schema.
60. **What is MLflow?** Experiment/run tracking of parameters, metrics, artifacts and model lineage.
61. **What is SHAP?** Model-output contribution allocation relative to a background, not proof of causal effect.
62. **What are drift types?** Data/covariate, prior/label and concept/conditional shifts.
63. **How demonstrate reliability?** Proper split, leakage audit, CV, independent test, uncertainty and failure analysis, monitoring plan.
64. **Can feature importance prove causation?** No; predictive association can come from proxy correlations or sampling artifacts.
65. **What if deployment performance drops?** Check schema/data collection changes, drift, delayed-label metrics and subgroups before retraining.

50. Defending your ML project — oral presentation framework

A clear 6–8 minute story

1. The business question (45 sec): "Who needs this prediction, when, and what decision will they make?" Define observation unit, target horizon, classification/regression type, operational cost.

2. Data understanding (60 sec): Data source, columns, sample count, class balance, date ranges, quality issues. Mention what could go wrong (duplicate customers, future-known columns, missingness).

3. EDA (60 sec): Show 2–3 meaningful plots only: target distribution, strongest *associations* or key missingness, and a potentially problematic feature. Say what decisions EDA justified.

4. Split and preprocessing (60 sec): Name exact validation strategy: stratified, grouped, temporal; imputation/encoding/scaling within pipeline. Explain why feature selection, SMOTE and PCA cannot be globally fit before CV.

5. Baseline and models (60 sec): State dummy score and two/three justified candidate models; explain expected strengths and costs. Avoid a wall of algorithms without comparison.

6. Evaluation, tuning and diagnostics (90 sec): Primary metric, secondary metric, cross-validation design, best parameters and uncertainty, confusion matrix or residual plot, which errors matter. If classes imbalanced, show precision/recall/PR and decision threshold.

7. Final result (45 sec): Untouched test metrics, evidence of generalization, shortcomings. Separate development CV from final test. Don't describe test as a dataset you "used to improve the model."

8. Deployment and next steps (30 sec): Serialize full pipeline; FastAPI/Streamlit; model version, privacy, monitoring and retraining triggers.

Practical response templates

Why this model? "I compared against a simpler baseline using the same validation protocol. This model offered [specific measurable benefit] on [primary metric] while retaining acceptable [latency/interpretability/fairness]. It was chosen before the test evaluation."

What is your preprocessing pipeline? "I separated numerical and categorical columns with ColumnTransformer; fitted imputation/encoding/scaling on each CV training fold; and applied the exact same fitted preprocessing to validation/test and production inputs."

How did you prevent data leakage? "I identified prediction-time information, excluded post-outcome fields, kept repeated entities grouped where needed, withheld final test data, and placed learned preprocessing and samplers in the cross-validated training pipeline."

Why this metric? "The cost of [false negatives, false positives or absolute errors] is higher in this application, so I chose [metric] and reported [second metric] to show the tradeoff."

What would you improve? "Collect more representative labels, validate across dates/groups, review calibration and drift, test operational impact and monitor actual outcomes before expanding deployment."

French vocabulary for soutenance

English	French	Key idea
Supervised learning	Apprentissage supervisé	Labeled outcomes
Unsupervised learning	Apprentissage non supervisé	No target labels
Training set	Ensemble d'entraînement	Learns parameters
Validation set	Ensemble de validation	Chooses models/tuning
Test set	Ensemble de test	Final independent evaluation
Feature engineering	Ingénierie des variables	Build predictors
Target / label	Variable cible / étiquette	Predicted outcome
Standard deviation	Écart-type	Dispersion
Data leakage	Fuite de données	Hidden information contamination
Overfitting	Surapprentissage	Fits training-specific noise
Underfitting	Sous-apprentissage	Too rigid to learn signal
Bias–variance tradeoff	Compromis biais–variance	Complexity/generalization tradeoff
Precision	Précision	TP/(TP+FP)
Recall	Rappel / sensibilité	TP/(TP+FN)
Specificity	Spécificité	TN/(TN+FP)
Confusion matrix	Matrice de confusion	TP/FP/FN/TN
Cross-validation	Validation croisée	Repeat validation folds
Hyperparameter tuning	Optimisation des hyperparamètres	Search model controls
Regression	Régression	Numerical target
Classification	Classification	Categorical target
Clustering	Regroupement / clustering	Unlabeled grouping
Decision boundary	Frontière de décision	Separator between classes
Loss function	Fonction de perte / coût	Penalizes mistakes
Gradient descent	Descente de gradient	Iterative optimization
Drift	Dérive des données / du concept	Distribution or relationship shifts
Pipeline	Chaîne de traitement	Reusable fitted workflow

51. Code review: spot the ML mistakes

Broken snippet A — scaler before split:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)  # LEAKAGE: all rows
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y)
model.fit(X_train, y_train)
```

Fix: Split first, place `StandardScaler()` inside `Pipeline` and fit only on train or each CV train fold.

Broken snippet B — SMOTE before CV:

```
X_bal, y_bal = SMOTE().fit_resample(X, y)   # leakage across folds
scores = cross_val_score(model, X_bal, y_bal, cv=5)
```

Fix: Use `imblearn.pipeline.Pipeline([('sampler', SMOTE()), ('model', model)])` on original development data and let CV perform resampling inside training folds only.

Broken snippet C — tune test set:

```
for depth in range(2, 21):
    model = DecisionTreeClassifier(max_depth=depth).fit(X_train,y_train)
    score = f1_score(y_test, model.predict(X_test))
    # pick maximum test f1 = invalid final score
```

Fix: Tune on development folds, then evaluate the chosen model once on sealed test.

Broken snippet D — use hard labels for ROC-AUC:

```
roc_auc_score(y_test, model.predict(X_test))
```

Fix: Pass continuous probability/decision scores for meaningful threshold-independent AUC; hard-label ROC-AUC is typically a degenerate summary.

Broken snippet E — collapse rare-positive data to accuracy:

```
print('Our accuracy is 99%, amazing!')
```

Fix: Show positive prevalence, dummy baseline, TP/FP/FN/TN, precision, recall, PR-AUC, threshold and expected costs.

Broken snippet F — input schema mismatch:

```
new_row = [[1000, 30]]      # training order was [age, income]
pipeline.predict(new_row)   # semantically swapped features
```

Fix: Save feature names, use a DataFrame/validated schema with correct column order, and test serialization + inference end-to-end.

Broken snippet G — time leakage:

```
df['next_week_spend'] = df.groupby('user')['spend'].shift(-1)
# Predict next-week purchase using next_week_spend as a current feature
```

Fix: Define prediction timestamp, use only lagged/history features with correct group, ordering and forecast horizon.

Broken snippet H — label encodes target itself:

```
X['closed_account'] = raw['closure_date'].notna().astype(int)
y = raw['has_churned']
```

Fix: Exclude target proxies unavailable at decision time; inspect every column's source and timestamp.

Broken snippet I — polynomial scaling outside CV:

```
X_pca = PCA(n_components=10).fit_transform(X)  # looks unsupervised but still leakage
cross_val_score(LogisticRegression(), X_pca, y, cv=5)
```

Fix: Fit PCA inside a `Pipeline` for each training fold.

Broken snippet J — probability threshold tuned on test:

```
threshold = max(np.arange(.1, .9, .05),
                key=lambda t: f1_score(y_test, model.predict_proba(X_test)[:,1] > t))
```

Fix: Tune threshold on development out-of-fold or validation predictions, freeze it, score test once.

Self-test: For each bug, explicitly state: which data are illegally used, how metrics become optimistic or unreliable, and what architecture prevents repetition. See exercises for variants and corrections in Chapter 53.

Chapter 52 — Exercises, challenges and mini-projects

How to study: Attempt these before opening `09_Solutions.md`. Work on paper for formulas, then verify using Python. For coding questions, keep leakage-free splits and explain each major design choice. Complexity rises from fundamental → applied → oral defense.

Level A — Foundations, probability and statistics

E01 · Identify task types. Classify each as regression, binary classification, multiclass classification, multilabel, clustering, anomaly detection or dimensionality reduction: (a) predict apartment price; (b) group users with no existing segment labels; (c) predict whether a customer will renew; (d) identify whether a comment is both spam and promotional; (e) detect unusual website request volume; (f) compress 150 correlated sensor features into 10 numeric components; (g) predict one of four flower types.

E02 · Shapes and dot product. X is shape `(800,12)`, w is `(12,)`, $b = 2$. What is the shape of `X @ w + b`? Calculate $w^\top x + b$ for $w = (2, -3)$, $x = (4, 5)$, $b = 7$.

E03 · Summary statistics. For `[2,4,4,4,5,5,7,9]`, calculate mean, median, population variance and population standard deviation. Which changes more under one gigantic new number, mean or median?

E04 · Boolean masking. For `A=np.array([[4,8,1],[9,2,7],[6,5,3],[1,8,9]])`, select rows whose first feature is >=6 and whose third is <8. Then compute per-column mean on the selected rows.

E05 · Bayes and prevalence. A detector has sensitivity 90%, specificity 95%; true event prevalence is 1%. Calculate $P(\text{event} \mid +)$, show formula. Why does it differ from 90%?

E06 · Gradient descent. If derivative $dJ/dw = 6$ at current $w = 2$ with learning rate `.1`, what is updated w? What if learning rate is 10? Explain why "bigger always faster" fails.

E07 · Sampling. A subscription dataset has multiple monthly rows per customer. A random row-wise 80/20 split puts customer IDs in both subsets. Is that a trustworthy test? Explain proper split and how the decision changes if objective is "predict next month for *existing* customers."

E08 · ML ethics and sources. A feature `cancelled_date` is present in a historical dataset for predicting cancellation next week. Can it be used? What about `days_since_last_login`, computed only up to current prediction timestamp?

Level B — EDA, preprocessing and validation

E09 · Missingness. A dataset has numeric `income` with large right-tail outliers and categorical `contract` with 6% missingness. Propose train-safe imputers, what to investigate first, and a way to retain missingness information.

E10 · Encoding. `contract={monthly,yearly,quarterly}` and `satisfaction={low,medium,high}`. Which encoding fits each, and why is `monthly=0,quarterly=1,yearly=2` potentially harmful to logistic/KNN?

E11 · Scaling. A KNN uses age in years (18–75) and annual income in currency units (15,000–250,000). Explain the distance failure and propose a pipeline fix. Would scaling be as crucial for a Random Forest?

E12 · Data leakage. Identify all mistakes: `df = read_csv(); X = StandardScaler().fit_transform(df.drop('target')); X,y=SMOTE().fit_resample(X,df.target); X_train,X_test,...=train_test_split(X,y); model.fit(X_train,y_train); report(test)`.

E13 · Split choice. Choose between stratified, group, time and regular K-fold for: (a) customer churn with rare positives, IID customers; (b) data recorded from patients repeatedly; (c) next-week demand prediction; (d) grouped device measurements with multiple sessions per device.

E14 · EDA interpretation. Correlation between ad spending and sales is +.9, but campaigns are increased before holidays and holidays increase sales. Can we infer that increasing ad spending causes more sales? What analyses would help?

E15 · Feature engineering. For customer events with date/timestamp, list four potential features allowed at the prediction date; explain one future leakage risk. Propose how to encode day-of-week cyclically (optional: hour-of-day).

E16 · Class imbalance. 5,000 examples contain 50 positives. Always predicting negative gives what accuracy and positive recall? Name three more informative metrics/diagnostics.

E17 · Pipeline diagram. Sketch a `ColumnTransformer` for `age`, `balance`, `contract`, `region`. Include numeric and categorical imputation, scaling where relevant, one-hot, and `LogisticRegression`. State where to put `SMOTE` if needed and how CV should fit it.

E18 · NumPy vs Pandas std. Explain why `np.std([1,2,3])` and `pd.Series([1,2,3]).std()` differ by default and which denominator each uses.

Level C — Regression and mathematical reasoning

E19 · Simple regression manually. Given $x = [1, 2, 3]$ and $y = [2, 4, 5]$, find least-squares slope and intercept using covariance/variance sums. Predict at $x = 4$.

E20 · Model errors. Actual `[3, -0.5, 2, 7]`, predictions `[2.5, 0, 2, 8]`: compute MAE, MSE, RMSE. State why MAPE may be a poor metric for target values near zero.

E21 · Negative R^2. On test labels `[1,2,3]`, model predicts `[10,10,10]`. Calculate R^2 and interpret. Does negative R^2 mean scikit-learn is broken?

E22 · Overfit polynomial. Training error decreases monotonically as polynomial degree rises 1→15, but CV error has minimum at 3. Which degree is preferable, and why? How should scaling, polynomial generation and Ridge tuning be organized?

E23 · Ridge vs Lasso. You have 250 highly correlated customer behavior predictors; interpretability matters but features are not truly sparse. Argue for Ridge vs Lasso vs Elastic Net, and give one CV hyperparameter each.

E24 · Compare four regressors. Use `load_diabetes`, 5-fold CV on development split and score MAE for `DummyRegressor`, `Ridge`, `RandomForestRegressor`, `HistGradientBoostingRegressor`. Use correct scaling for Ridge, a fixed split, same folds. Report mean/std CV MAE and final MAE for selected model; do not tune with final test.

E25 · Diagnosing regression plots. Residual plot curves upward, variance fans out at high predictions, and some points have large influence. Identify three modeling concerns and one response for each.

E26 · SVR details. Explain `C`, `epsilon`, `gamma` and `kernel='rbf'` without confusing them with probabilities. Why scale the features?

E27 · Bagging vs boosting (regression). Compare fitting process, parallelism, regularization and sensitivity to sequential error correction. Why does boosting often need a smaller learning rate with more trees?

Level D — Classification and metrics

E28 · Confusion matrix by hand. TP=40, FP=10, FN=20, TN=130. Compute accuracy, precision, recall, specificity, F1, FPR and balanced accuracy. State what extra information you need to choose a real decision threshold.

E29 · Rare class. A binary classifier gets accuracy .98, positive precision .20 and recall .10 on a dataset where only 1% churn. Is it good? What baseline and metrics would you request? Are those reported numbers necessarily mathematically consistent for the given prevalence?

E30 · Threshold decision. Two thresholds on the same validation scores: `t=.3 → TP=80, FP=50, FN=20, TN=850` and `t=.7 → TP=45, FP=10, FN=55, TN=890`. Compute precision and recall at each; choose threshold if missing a case costs 10× a false alarm using $10FN + FP$.

E31 · ROC-AUC vs PR. Why might ROC-AUC be high when only .5% of observations are positive, while precision is poor? Explain AP vs trapezoidal PR-AUC.

E32 · Probability calibration. Model predicts probability .8 for 100 samples; only 40 turn out positive. What does this suggest? Can it still have high ROC-AUC? How would you validate/calibrate without test leakage?

E33 · Logistic coefficient. Logistic coefficient for `tenure_years` is -0.7. Calculate associated odds multiplier for +1 year and explain why this is not "70 percentage points fewer churn."

E34 · Tree impurity. A node has class proportions .6 positive and .4 negative. Compute Gini impurity and binary entropy. Why does a pure child have impurity zero?

E35 · SVM hyperparameters. On a small, scaled nonlinear dataset, RBF SVM overfits badly. Explain how adjusting `C` and/or `gamma` may help, and how to select combinations without test leakage.

E36 · Multiclass/multilabel. Identify type: (a) one of dog/cat/bird; (b) image can have multiple tags including indoor and night; (c) churn yes/no. Explain `macro` vs `weighted` F1; name a case where macro better communicates minority performance.

E37 · Choose algorithm. You need a baseline on 30k sparse TF-IDF documents and 3 classes. Compare MultinomialNB, LinearSVC, LogisticRegression, RBF SVC and KNN on computation/interpretability/fit; propose two to test first.

Level E — Clustering, PCA and anomalies

E38 · Silhouette. For a point, within-cluster distance $a = 2$ and nearest-other-cluster average distance $b = 5$. Compute silhouette. What if $a = 6$, $b = 3$?

E39 · K-Means. Why does inertia always improve or stay equal when k increases (assuming solutions not worse)? Why can't it choose k alone? How can scaling change customer segments?

E40 · DBSCAN. For `eps=.2`, all but 5 points become noise; for `eps=3` everything is one giant cluster. What do these observations imply? Which diagnostic or alternate method could help if density varies strongly?

E41 · PCA / SVD. PCA components explain variance fractions `[.45,.25,.15,.08,.07]`. How many components to retain at least 90%? Why is it wrong to say "90% of target information retained"?

E42 · GMM. Distinguish responsibilities, centroids, covariance and mixture weights. Why could a GMM model two elongated groups where K-Means struggles? What are AIC/BIC for?

E43 · Anomaly. An IsolationForest with `contamination=.05` marks 5% as anomalies. Does this prove 5% are fraud? Explain how to evaluate and what to monitor.

Level F — Professional ML and debugging

E44 · Grid search complexity. `C=[.1,1,10,100]`, `gamma=[.001,.01,.1]`, `kernel=['rbf']`, 5-fold CV, `refit=True`: how many model fits, ignoring any other setup? If 5-fold nested outer CV re-runs this search, how many inner fits and outer final fits?

E45 · Model selection bias. You compare 50 hyperparameter configurations and report the best mean CV F1 as final test F1. Identify what's misleading and propose two more honest evaluation designs.

E46 · Correct the bug. Write safe pseudocode for `X_scaled=StandardScaler().fit_transform(X); PCA.fit_transform(X_scaled); cross_val_score(LogisticRegression(), X_pca, y, cv=5)`.

E47 · Persistence. You saved only `RandomForestClassifier` but not the fitted `OneHotEncoder`. Describe a likely production failure and correct artifact strategy. Explain why loading arbitrary `.joblib` files is unsafe.

E48 · Interpretability. A Random Forest says `customer_id` has strongest impurity importance. What could be happening? How would you investigate with permutation importance, target leakage review, and grouping?

E49 · MLflow. Specify which metadata you would log when comparing logistic regression, Random Forest and XGBoost on churn, and which fields you would show your soutenance jury.

E50 · Drift. After deployment, fraction of monthly contracts doubles, labels arrive 30 days late, and prediction latency rises sharply. Which monitor for each issue? Does every drift require retraining?

E51 · Oral defense. Give a 90-second defense of your churn pipeline using six required facts: task/target, data/split, preprocessing, baseline/candidates, metric/tuning, limitations/monitoring.

E52 · Debug threshold. Code tries thresholds on `y_test` and reports the maximum test F1. Rewrite the steps, including out-of-fold validation probabilities and refit/test.

Capstone mini-projects (graded by reasoning, not headline score)

Project P1 — House price / car price regression

Deliverables: EDA report (target units, skew, missingness), separate test, preprocessing, dummy baseline, Linear/Ridge, RandomForest and gradient boosting with 5-fold CV, MAE/RMSE/R^2, residual plots by price range, permutation importance, saved full pipeline, 3–5 minute oral defense. **Key question:** Why are MAE and RMSE different in the presence of rare expensive properties?

Project P2 — Customer churn classification

Deliverables: Data dictionary and prediction horizon; categorical/numeric pipeline; stratified or group/time split as needed; dummy/logistic/RF/boosting; class balance and calibration; CV AP/ROC-AUC/F1 and threshold-specific confusion matrix; threshold cost analysis; held-out test; joblib artifact; optional MLflow/Streamlit. **Key question:** What if target is observed 30 days after features?

Project P3 — Customer segmentation

Deliverables: Justify business feature set; explore missingness/skew; scaled K-Means for k 2…8, compare silhouette/inertia; DBSCAN and GMM comparison; PCA 2D projection with careful caveats; cluster profiles in original units; stability and practical action plan. **Key question:** Why can mathematically separated clusters still be a poor business solution?

Project P4 — Anomaly detection and deployment

Deliverables: Define rare event and review capacity; baseline rules, IsolationForest vs LOF; evaluate if verified labels available; document threshold-selection caveats; schema validation; simple FastAPI `predict` endpoint and Streamlit dashboard; monitoring plan for drift/error/latency. **Key question:** Why is anomaly not equal to malicious behavior?

Self-grading rubric (100 pts)

Category	Points	Excellent answer
Problem framing/data validity	15	Defines target, horizon, observation unit, sampling risks
EDA and feature design	15	Statistical evidence + thoughtful features, no leakage
Validation methodology	20	Correct split, fold-safe pipeline, untouched test
Models/metrics	20	Baselines, justified comparison, error/cost tradeoff
Reproducible implementation	15	Clean code, full artifact, tracked parameters/schema
Oral defense	15	Explains intuition, math, limitations and alternatives

Study order: E01–E18 → Part III and E19–E27 → Part IV and E28–E37 → Part V and E38–E43 → Part VI and E44–E52 → one capstone → oral simulation. Corrections are in Chapter 53.

Chapter 53 — Worked Solutions and Corrections

Do the exercises first. A good written answer includes *why* a technique is appropriate, not just a final number or a library call. The numbered solutions follow Chapter 52.

Level A — Foundations

E01. (a) Regression, (b) clustering, (c) binary classification, (d) multilabel classification, (e) anomaly detection, (f) dimensionality reduction, (g) multiclass classification. Reason: task type comes from the **output we want** and availability of labels, not the data file format.

E02. `(800,)`: 800 samples each with one numeric prediction. Dot product $2(4)+(-3)(5)+7 = 0$. With intercept broadcast, the number of rows does not change.

E03. Sum 40, $n = 8$, mean 5. Sorted middle values 4 and 5 → median 4.5. Squared deviations from 5 are `[9,1,1,1,0,0,4,16]`, sum 32, population variance $32/8 = 4$, population std 2. Mean is heavily pulled by an enormous new observation, median changes far less.

E04. Rows are `[9,2,7]` and `[6,5,3]`; first >=6 and third <8. Per-column mean `[7.5, 3.5, 5]`.

```
import numpy as np
A = np.array([[4,8,1],[9,2,7],[6,5,3],[1,8,9]])
selected = A[(A[:,0]>=6)&(A[:,2]<8)]
print(selected, selected.mean(axis=0))
```

E05. $P(+) = .90(.01) + .05(.99) = .0585$; $P(\text{event} \mid +) = .009/.0585 \approx 0.153846 = 15.38\%$. Base rate is small, so false alarms from the large negative population dominate.

E06. $w_{new} = 2 - .1(6) = 1.4$. With LR 10, $w_{new} = 2 - 60 = -58$. Overshooting optimum may cause divergence/oscillation; learning-rate correctness depends on loss curvature and scale.

E07. Row-wise random split violates independence; repeated customers allow leakage/memorization. For *new customer* generalization use group holdout by ID. For *future of existing customers*, preserve chronological order and prediction horizons, with grouped/blocked temporal CV as needed; choose split that matches intended deployment.

E08. `cancelled_date` is a target proxy unavailable at prediction time; drop it (unless prediction occurs *after* cancellation, in which case task must be redefined). `days_since_last_login` is valid only if computed from logs known up to prediction time and correctly handles late arrival.

Level B — Data preparation

E09. Investigate missingness against time, groups and target to see whether it is systematically missing. Train-fold median imputation for skewed `income` is a reasonable baseline; most-frequent or constant `Unknown` for `contract`. Use `SimpleImputer(add_indicator=True)` on numeric inputs or a separate missing flag. Compare effect by CV, not by test fit.

E10. One-hot for nominal contract; ordinal mapping low=0, medium=1, high=2 if order meaningful (note arbitrary equal spacing for some models). Assigning 0/1/2 to nominal categories imposes fake distances and a monotonic coefficient interpretation on logistic/KNN.

E11. Raw income differences overshadow age in Euclidean distance because currency is numerically huge. Use `Pipeline([('impute',...),('scale',StandardScaler()),('knn',...)])` and fold-safe fitting; forests generally don't need scaling because split ordering is preserved.

E12. Scaling on all rows leaks test statistics; SMOTE on all rows leaks/synthetically duplicates neighborhoods across future split; sample prevalence altered in test and metric becomes nonrepresentative. Split original rows first (with appropriate groups/time); place scaling + SMOTE + estimator in `imblearn.pipeline.Pipeline`; cross-validate on development rows and evaluate original held-out test distribution.

E13. (a) StratifiedKFold (for IID customers), (b) group-aware split by patient, (c) chronological/TimeSeriesSplit with horizon gap, (d) grouped by device. For rare positives and groups, consider StratifiedGroupKFold where feasible. Split selection must reproduce deployment.

E14. No causality follows from correlation; holiday is a confounder. Investigate time controls, seasonality, quasi-experiments, randomized campaigns if ethical/feasible, lagged effects and external changes. An ML model may still use correlation for prediction without causal claims.

E15. Allowed up to prediction moment: tenure, transactions last 30 days, days since last activity, known contract type. Leakage: `next_month_total_spend` or future rolling window. Hour cyclical encoding `sin(2*pi*hour/24)`, `cos(2*pi*hour/24)`; weekday `sin(2*pi*dow/7),cos(2*pi*dow/7)`.

E16. 4950 negatives, 50 positives: all-negative accuracy $4950/5000 = .99 = 99\%$; positive recall 0. Examine precision, recall, F1, average precision/PR curves, confusion matrix, specificity, business costs and threshold performance.

E17. `ColumnTransformer([('num', Pipeline([median_imputer, StandardScaler]), ['age','balance']), ('cat', Pipeline([most_frequent_imputer,OneHotEncoder(handle_unknown='` `['contract','region'])])) → LogisticRegression.` If resampling needed use `imblearn.pipeline.Pipeline([('pre', pre),('smote', SMOTE()),('clf', ...)])` and beware synthetic one-hot categories (consider `SMOTENC` at appropriate stage or class weights). Every CV fold must fit imputer/encoder/scaler/sampler on its training subset only.

E18. `np.std` defaults to `ddof=0` and divisor n; `pd.Series.std` defaults `ddof=1` and divisor $n-1$. For `[1,2,3]`: NumPy $\sqrt{2/3} \approx 0.8165$, Pandas 1.

Level C — Regression

E19. $\bar{x} = 2, \bar{y} = 11/3$. Numerator $\sum(x_i - 2)(y_i - 11/3) = (-1)(-5/3) + 0 + (1)(4/3) = 3$. Denominator $(-1)^2 + 0 + 1^2 = 2$; slope 1.5; intercept $11/3 - 1.5(2) = 2/3$. Prediction at 4: $2/3 + 1.5(4) = 20/3 \approx 6.667$.

E20. Absolute errors `[.5,.5,0,1]` → MAE=.5. Squared errors `[.25,.25,0,1]` → MSE=.375. RMSE=$\sqrt{.375} \approx .61237$. MAPE divides by true label, so zero/near-zero truth causes severe instability; negative/zero target makes interpretation awkward.

E21. $\bar{y} = 2$. SSE=$(1-10)^2 + (2-10)^2 + (3-10)^2 = 81 + 64 + 49 = 194$. SST=$(1-2)^2 + 0 + (3-2)^2 = 2$. R^2=$1 - 194/2 = -96$. This model is much worse than predicting mean of test targets; negative R^2 is permitted.

E22. Degree 3 has lowest CV error, so select on CV not training score. Put `PolynomialFeatures → StandardScaler → Ridge` inside a pipeline, grid-search degree/alpha on development folds, then test once. High-degree models fit noise and extrapolate wildly.

E23. Ridge stabilizes correlated effects and preserves them; Lasso selects sparse predictors but can choose unstable members among correlated features; Elastic Net combines them. Tune `ridge__alpha`, `lasso__alpha`, `elasticnet__alpha` plus `elasticnet__l1_ratio` via CV (correct pipeline names depend on steps). Use standardized inputs and repeated folds/stability inspection for feature selection.

E24. Reference implementation using `load_diabetes` and correct CV, not a fixed promised score:

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, KFold, cross_validate
from sklearn.dummy import DummyRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor, HistGradientBoostingRegressor
from sklearn.metrics import mean_absolute_error
X,y = load_diabetes(return_X_y=True)
Xd,Xt,yd,yt=train_test_split(X,y,test_size=.2,random_state=42)
folds=KFold(n_splits=5,shuffle=True,random_state=42)
models={
    'dummy':DummyRegressor(),
    'ridge':make_pipeline(StandardScaler(),Ridge(alpha=1.0)),
    'forest':RandomForestRegressor(n_estimators=150,min_samples_leaf=3,random_state=42),
    'hgb':HistGradientBoostingRegressor(max_iter=150,random_state=42),
}
results={}
for name,m in models.items():
    cv=cross_validate(m,Xd,yd,cv=folds,scoring='neg_mean_absolute_error')
    maes=-cv['test_score']; results[name]=maes.mean()
    print(name, 'mean:',round(maes.mean(),2),'std:',round(maes.std(),2))
winner=min(results,key=results.get)
final=models[winner].fit(Xd,yd)
print('selected:',winner, 'test MAE:',mean_absolute_error(yt,final.predict(Xt)))
```

E25. Curved residual trend → unmodeled nonlinear relation (engineer interactions, flexible model). Fan-shaped residual variance → heteroscedasticity (target transform, quantile/robust

objective, segment analysis). Influential points → check data validity, leverage and robust alternatives; don't delete legitimate rare cases by default.

E26. C penalizes violating epsilon tube; epsilon margin of tolerated regression errors; gamma scale of RBF similarity (high means local influence); RBF is nonlinear similarity kernel. Scaling prevents unrelated units dominating kernel distances. Tune on CV, not test.

E27. Bagging trains trees on random training subsamples and aggregates, can train many in parallel; boosting trains sequential corrective stages minimizing loss and controls complexity via shrinkage and depth. With smaller step size, more boosting iterations are often needed to approximate the same fit and overfitting may reduce. Neither is universally superior.

Level D — Classification

E28. $N = 200$. Accuracy=$170/200 = .85$. Precision=$40/50 = .80$. Recall=$40/60 = 2/3 = .6667$. Specificity=$130/140 = 13/14 = .92857$. F1=$2TP/(2TP + FP + FN) = 80/(80 + 10 + 20) = 8/11 \approx .72727$. FPR=$10/140 = 1/14 \approx .07143$. Balanced accuracy=$(2/3 + 13/14)/2 = 67/84 \approx .79762$. Need error costs, intervention limits, calibration/score distributions and validation context to choose threshold.

E29. Accuracy .98 sounds high but a 99% negative dummy exceeds it. Positive precision .20 and recall .10 suggest a poor actionable minority classifier (depending on costs). Request confusion matrix, class prevalence, AP, ROC-AUC, calibrated probabilities, threshold and confidence intervals. **Consistency check:** $N = 10000$, positives 100, TP=10 from recall .1, FP=40 from precision .2, FN=90, TN=9860. Accuracy $(10 + 9860)/10000 = .987$, not exactly .98; stated figures may be rounded/aggregated across datasets, so demand sample counts and averaging method. (Other consistent prevalence/recall/precision combinations similarly imply about 98.7% accuracy.)

E30. At $t = .3$: precision=$80/(80+50) = .6154$, recall=$80/100 = .8$, cost=$10(20)+50 = 250$. At $t = .7$: precision=$45/(45 + 10) = .8182$, recall=$45/100 = .45$, cost=$10(55) + 10 = 560$. Under defined costs prefer .3, based **only on validation**; do not tune threshold with final test.

E31. ROC-FPR divides FP by huge negatives; small FPR can still imply many false positives with very low base rate. PR precision includes these false positives directly. `average_precision_score` computes a weighted precision summary over recall steps; trapezoidal `auc(recall, precision)` linearly interpolates points and can differ. Report exact metric definition.

E32. Forecasted probability .8 but observed positive proportion .4 in that bin suggests overconfidence/miscalibration, assuming enough samples and representative bin. Model may still rank positives well (high ROC-AUC). Use `CalibratedClassifierCV` on development folds; assess reliability on a held-out validation portion and final untouched test after decisions fixed.

E33. Odds multiplier $e^{-0.7} \approx 0.4966$ per additional tenure year, holding predictors fixed. Odds roughly halve, **not** probabilities nor absolute percentage points. Probability impact depends on baseline odds.

E34. Gini=$1 - (.6^2 + .4^2) = 1 - .52 = .48$. Entropy=$-.6 \log_2(.6) - .4 \log_2(.4) \approx .97095$ bits. For a pure node ($p = 1$ or 0), no label uncertainty: both impurities 0.

E35. Reduce C to strengthen regularization or reduce gamma for smoother RBF influence; potential underfitting if pushed too far. Tune grid of both via stratified or group-safe cross-validation inside scaling pipeline on development data, not test.

E36. (a) Multiclass, (b) multilabel, (c) binary. Macro-F1 averages classwise F1 equally and shows minority weakness; weighted-F1 weights by support and may be dominated by majority. For rare medically relevant minority (hypothetical educational context), macro makes poor minority performance harder to hide, but choose actual costs and monitor individual classes.

E37. Test TF-IDF + MultinomialNB, TF-IDF + LinearSVC (or logistic regression). They scale well to high-dimensional sparse matrices; logistic yields probabilities (subject to calibration). RBF SVC and KNN are usually more computationally expensive/less apt for very high-dimensional sparse text. Use stratified folds and a metric informed by class balance.

Level E — Unsupervised

E38. If $a = 2, b = 5$: $(5 - 2)/5 = .6$. If $a = 6, b = 3$: $(3 - 6)/6 = -.5$ → may be closer to another cluster than its own.

E39. Allowing more centroids can replicate the old solution and optionally reduce distances; globally optimal inertia cannot increase with k. Since inertia tends to decrease monotonically, use elbow, silhouette, stability, balanced segment sizes and business usefulness. Unscaled currency will dominate age/behavior distances, changing assignments.

E40. Small eps → disconnected sparse neighborhoods and many noise labels; huge eps → merged clusters. Inspect k-distance plot, standardized units, min_samples, neighborhood densities, try multiple eps; with varying densities consider HDBSCAN or hierarchical methods. High dimensionality can make distance unreliable.

E41. Cumulative explained variance: 0.45, 0.70, 0.85, 0.93 → **4 components** to reach 90%. PCA preserves unsupervised variance, not target-label predictability.

E42. **Responsibilities:** posterior soft membership probabilities per row. **Means/centroids:** center of each Gaussian. **Covariance:** size, elongation, orientation. **Mixture weights:** prior share of components; sum to 1. Full covariance lets components elongate; K-Means uses compact Euclidean Voronoi partitions. AIC/BIC compare penalized likelihood of fitted models, not ground truth clusters.

E43. No: `contamination=.05` is a threshold/expected outlier setting, not fraud prevalence. If verified cases exist, report precision/recall at review capacity, subgroup false-alarm rate and cost on appropriately held-out time slices; otherwise manual review, stability and domain validation. Monitor feature drift, alerts and reviewed outcomes.

Level F — Professional workflow

E44. Grid points 4×3×1=12; CV fits 12×5=60, plus one `refit=True` on full development → **61 fits**. Nested with five outer folds, each inner search does 60 CV fits plus one refit for that outer fold: **305 fits total** (5×61), assuming same grid sizes and 5 inner folds, plus optional final refit on all development data if needed. If counting **inner CV fits and outer final fits separately**, inner =300 and outer refits =5.

E45. Best CV F1 is selected with noise and optimistically biased as a final estimate; not an untouched test. Design A: reserve final test and tune on development folds, then evaluate once. Design B: nested CV with outer unbiased workflow evaluation (plus final fit for deployment). Confidence intervals/temporal external validation enhance evidence.

E46. Correct pattern:

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
pipe=make_pipeline(StandardScaler(),PCA(n_components=10),
                   LogisticRegression(max_iter=2000))
cv=StratifiedKFold(n_splits=5,shuffle=True,random_state=42)
score=cross_val_score(pipe,X_dev,y_dev,cv=cv,scoring='balanced_accuracy')
# Fit final selected pipeline on development data; evaluate untouched X_test once.
```

`PCA(n_components=10)` requires at least 10 appropriate features and fold training samples; adjust to dataset.

E47. New category/order differs from fitted one-hot columns; standalone forest receives wrong shape or semantics. Save full fitted `Pipeline` and schema plus dependency versions; test inference. Pickle/joblib deserialization may execute arbitrary code, so load only trusted model artifacts.

E48. `customer_id` may encode time, segment, repeated people, labels, high cardinality or data leakage. Re-run without identifier and with proper group/time split, inspect row duplicates, compare CV, compute held-out permutation importance carefully; do not claim causality.

E49. Experiment/run name; code commit/version; training data snapshot/hash and date; target horizon; class balance; group/time folds; preprocessing version; seed; model family and hyper-

parameters; validation AP/AUC/F1 and confusion matrix at threshold; calibration; runtime; model artifact; limitations. Show side-by-side CV metrics and selected model reason in soutenance.

E50. Contract shift: monitor category distribution, proportions, unseen categories and subgroup metrics (covariate drift). Delayed labels: track pending labels and report performance only for matured cohorts, with time labels. Latency: p50/p95/p99 and error rate logs. Drift itself triggers investigation; retrain only if real performance/quality change and safe data allow it.

E51. Sample defense (adapt numbers to your real project): "I predict whether each active customer will churn within 30 days using only information available at the weekly prediction snapshot. I grouped customer histories and used a temporal holdout to simulate future data. My ColumnTransformer applies median imputation and scaling to numerical features, and one-hot encoding to categorical features, learned within each training fold. I compared a dummy baseline, logistic regression, a random forest and boosted trees under the same grouped validation. I selected average precision as the main ranking measure given rare churners; I also examined recall, precision, calibration and a threshold chosen on validation alerts. I then evaluated the locked pipeline once on the final test period, inspected segment errors and saved the full pipeline with schema/version information. Remaining risks are delayed churn labels, seasonal drift and policy-dependent thresholds; I would monitor them before deployment."

E52. (1) Keep test untouched. (2) Split development by correct groups/time. (3) For candidate model, collect out-of-fold *scores* using CV; be careful that each fold's score is produced by a model not trained on that row. (4) Choose threshold on out-of-fold scores or separate validation data under operational cost. (5) Fix model params and threshold. (6) Fit final pipeline on all development data. (7) Apply frozen threshold to test scores, report once. (8) Track test data use to avoid repeated selection.

Capstone review guide

P1 Regression: Prefer realistic price units, MAE and tail RMSE. Check temporal/geographic splits for house price; compare Ridge and tree models and inspect if best model fails on expensive listings. A 5% difference in MAE may matter less than severe errors in specific regions.

P2 Churn: Use positive label `churn=1`, calculate prevalence and dummy scores, avoid late outcomes as features, train grouped/temporal if observations repeat, and tune thresholds to business capacity. Report AP and threshold-specific precision/recall, not just accuracy.

P3 Segmentation: Business interpretation comes from original feature distributions, not arbitrary cluster ID. Compare K-Means/GMM/DBSCAN and stability; justify number of groups with utility and sizes, not solely elbow.

P4 Anomaly: Fraud/risk labels need domain verification. IsolationForest predicts statistical unusualness; make clear what false positives cost, and monitor schema drift, human-review outcomes and sensitivity to contamination threshold.

Best soutenance habit: When asked "Why?", distinguish **design constraint**, **mathematical property**, **observed validation evidence**, and **remaining uncertainty**. That is stronger than memorizing an algorithm list.

Chapter 54 — Final Revision Cheat Sheets

For the final hour before a soutenance croisée. Detailed explanations are in Parts I–VII. Key answer format: *What is it? → How does it work? → When use it? → What could go wrong? → How would I evaluate?*

The ML lifecycle in 12 verbs

Define → Audit → Split → Explore → Engineer → Pipeline → Baseline → Compare → Tune → Diagnose → Test → Deploy/Monitor.

Gold rule: The model, preprocessing, feature selection, resampling and threshold must never learn from test observations/labels. Group/time structure determines split strategy.

Task-to-algorithm lookup

Problem	Baseline	Main candidates	Key metric
Continuous numeric target	Dummy mean/median	Linear, Ridge, RF, SVR, boosting	MAE, RMSE, R^2
Binary label	Dummy majority/prior	Logistic, RF, SVM, XGBoost	Precision, recall, F1, AP, ROC-AUC
Multiclass label	Dummy	Multinomial logistic, RF, boosting	Macro F1, log loss, accuracy
Sparse text classification	Majority	TF-IDF + NB/LinearSVC/logistic	Macro F1, AP where relevant
Segmentation	Descriptive baselines	K-Means, GMM, DBSCAN, hierarchical	Stability, silhouette, business use
Dimensionality reduction	None	PCA, SVD; t-SNE/UMAP for visualization	Explained variance / visual structure
Unusual-case detection	Domain rule	Isolation Forest, LOF, One-Class SVM	Precision/recall at alert capacity if labels exist

Key equations — regression

$$\hat{y} = \beta_0 + \sum_j \beta_j x_j; \quad e_i = y_i - \hat{y}_i.$$

$$MAE = \tfrac{1}{n}\sum |e_i|; \quad MSE = \tfrac{1}{n}\sum e_i^2; \quad RMSE = \sqrt{MSE}.$$

$$R^2 = 1 - \frac{\sum e_i^2}{\sum (y_i - \bar{y})^2}; \quad R^2_{adj} = 1 - (1-R^2)\frac{n-1}{n-p-1}.$$

$$MAPE = \tfrac{1}{n}\sum |e_i / y_i| \quad (\text{unstable at/near } y_i = 0).$$

Ridge: L2 penalty $\alpha\|\beta\|_2^2$; **Lasso:** L1 penalty $\alpha\|\beta\|_1$; **ElasticNet:** both. Coefficient meanings are conditional on included predictors and units. Ridge shrinks; Lasso can select zero weights.

Key equations — classification

$$z = w^\top x + b, \qquad P(y = 1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Metric	Formula
Accuracy	$(TP + TN)/(TP + TN + FP + FN)$
Precision	$TP/(TP + FP)$
Recall / TPR	$TP/(TP + FN)$
Specificity / TNR	$TN/(TN + FP)$
FPR	$FP/(FP + TN)$
F1	$2TP/(2TP + FP + FN)$
Balanced accuracy (binary)	$(TPR + TNR)/2$
ROC-AUC	Area under TPR vs FPR threshold curve
Average precision	Precision weighted by recall increases
Binary log loss	$-\frac{1}{n}\sum[y \ln p + (1-y)\ln(1-p)]$

Threshold: Smaller threshold often increases recall, at cost of false positives. Choose on validation/CV, never optimize after examining final test. AUC ranking ≠ calibration. Precision is prevalence-dependent. **Label meaning**: check `model.classes_` before `[:,1]`.

Key equations — unsupervised

$$\text{Inertia} = \sum_i \|x_i - \mu_{c_i}\|^2; \quad \text{Silhouette}(i) = \frac{b_i - a_i}{\max(a_i, b_i)}.$$

K-Means: k groups, compact centroid distance; initialize with K-Means++ and multiple starts. **DBSCAN:** eps, `min_samples`, noise label -1, variable-density challenge. **GMM:** soft Gaussian membership, covariance matters. **PCA:** orthogonal directions maximizing **input** variance; not necessarily target predictiveness. **t-SNE/UMAP:** visualizations where distances between islands should be interpreted cautiously.

Model parameters that examiners ask about

Model	Main controls	Larger means…
KNN	`n_neighbors`	Smoother/locality less detailed
Decision Tree	`max_depth`	More splits/complexity
Random Forest	`n_estimators`	More trees, usually stabler and slower
Random Forest	`min_samples_leaf`	Smoother, fewer tiny noisy leaves
Logistic Regression	`C`	Weaker regularization
SVM RBF	`C`	Stronger penalty on margin violations
SVM RBF	`gamma`	More local nonlinear influence
Ridge/Lasso	`alpha`	Stronger shrinkage
SVR	`epsilon`	Wider loss-free tolerance tube
Boosting	`learning_rate`	Larger step each stage, potentially more overfit
Boosting	`n_estimators`	More additive stages
K-Means	`n_clusters`	More centroids, lower inertia possible
DBSCAN	`eps`	Larger density neighborhoods / merged groups
PCA	`n_components`	More axes and preserved variance

Quick, safe Python recipes

Classification

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score
X_dev, X_test, y_dev, y_test = train_test_split(
    X, y, test_size=.2, random_state=42, stratify=y)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
print(cross_val_score(model, X_dev, y_dev, cv=5, scoring='balanced_accuracy').mean())
model.fit(X_dev, y_dev)
print(classification_report(y_test, model.predict(X_test)))
print(roc_auc_score(y_test, model.predict_proba(X_test)[:, 1]))
```

Regression

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score
Xd, Xt, yd, yt = train_test_split(X, y, test_size=.2, random_state=42)
model = make_pipeline(StandardScaler(), Ridge(alpha=1))
```

```
print((-cross_val_score(model, Xd, yd, cv=5,
                        scoring='neg_mean_absolute_error')).mean())
model.fit(Xd, yd)
pred = model.predict(Xt)
print(mean_absolute_error(yt,pred), root_mean_squared_error(yt,pred), r2_score(yt,pred))
```

Pipeline with categorical data

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
pre = ColumnTransformer([
    ('num', Pipeline([('imp', SimpleImputer(strategy='median')),
                      ('scale', StandardScaler())]), numeric_cols),
    ('cat', Pipeline([('imp', SimpleImputer(strategy='most_frequent')),
                      ('onehot', OneHotEncoder(handle_unknown='ignore'))]), categorical_cols)
])
model = Pipeline([('pre', pre), ('estimator', LogisticRegression(max_iter=2000))])
```

Cross-validation / tuning

```
from sklearn.model_selection import GridSearchCV
search = GridSearchCV(model,
    {'estimator__C':[0.1,1,10]},
    cv=5, scoring='average_precision', n_jobs=-1)
# search.fit(X_dev, y_dev)  # NEVER tune on X_test/y_test
```

Save/reload a pipeline (trusted files only)

```
import joblib
joblib.dump(model, 'complete_model.joblib')
# restored = joblib.load('complete_model.joblib')
```

Error-pattern quick diagnosis

What you observe	First suspicion	Fix/diagnostic
Accuracy 99%, recall 0	Severe imbalance	Baseline, confusion matrix, PR
Train 100%, CV 60%	Overfit / leakage / nonrepresentative folds	Audit splits, regularize
CV high, future test low	Time drift / leakage	Temporal evaluation, inspect timestamps
KNN is nonsense	Unscaled irrelevant features	Scale/train-fitted, feature selection
SVM low performance	Scaling, C/gamma	Proper pipeline and CV tuning
Tree has huge importance on ID	High cardinality / leakage	Drop/audit ID, group split
RMSE large vs MAE	Very large errors	Inspect residual tail/groups
K-Means bad segments	Wrong distance/scaling/k	Feature selection, test alternatives
Inference shape mismatch	Encoder/schema version drift	Save full pipeline, validate DataFrame schema
Probability .9 but 50% correct	Calibration problem	Reliability diagram, calibration folds

10 sentences worth memorizing

1. “The split must represent the way the model will be used.”
2. “I chose the metric based on the cost of errors.”
3. “All learned preprocessing lives inside the cross-validated pipeline.”
4. “I compare to a dummy baseline and a simple interpretable model.”

5. “I keep the final test set untouched until choices are locked.”
6. “Precision measures correctness of positive alerts; recall measures coverage of actual positives.”
7. “AUC measures discrimination; calibration measures probability reliability.”
8. “Random Forest averages diverse trees; boosting builds sequential corrections.”
9. “Clustering labels are not ground-truth categories or automatically actionable.”
10. “A useful model includes monitoring, limitations and a reproducible artifact.”

90-second exam readiness checklist

- ☐ Can I describe my target/horizon, grouping, number of rows and class balance?
- ☐ Can I show the exact pipeline and justify its encoders/scalers?
- ☐ Can I derive and interpret regression or classification metrics by hand?
- ☐ Can I explain CV model selection and final test separation?
- ☐ Can I justify why my chosen model beats simpler baselines?
- ☐ Can I state three limitations and a plan to monitor them?

For detailed corrections, see Chapter 53.

Appendix — Other Classical ML Topics Worth Recognizing

The 54-chapter curriculum covers the main requested algorithms. This appendix adds **non-deep-learning** domains that a jury may mention, without pretending that every research method belongs in a single guide.

A. Ensemble voting, bagging and stacking

Voting: `VotingClassifier(voting='hard')` majority vote of predictions, or `voting='soft'` average of estimated probabilities (only meaningful when estimators implement probabilities and calibration has been considered). **Bagging:** fit base estimators on randomly sampled rows/features; average/majority reduces variance. `BaggingClassifier`, `BaggingRegressor` can wrap more than decision trees. **Stacking:** train multiple base estimators, then a **meta-estimator** on cross-validated predictions, avoiding training-on-own-predictions leakage. `StackingClassifier`/`StackingRegressor` implement out-of-fold stacking logic; careless manual stacking overfits.

```
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
voter = VotingClassifier([
    ('linear', LogisticRegression(max_iter=1000)),
    ('forest', RandomForestClassifier(n_estimators=100,random_state=42))],
    voting='soft')
# Put scaling/encoding in each estimator pipeline when needed.
```

B. Robust, count, quantile and generalized linear regression

- **HuberRegressor:** robust linear objective, squared for small residuals and linear in large residuals; robust to some target outliers but not arbitrary influential leverage points.
- **RANSACRegressor:** fit candidate model on random subsets, identify consensus inliers; useful under a high proportion of gross-outlier observations if the base assumptions fit.
- **QuantileRegressor / GradientBoostingRegressor(loss=‘quantile’):** predict target conditional quantiles (e.g., 10th, 50th, 90th). Conditional quantiles can support risk bounds, but *not automatically calibrated* prediction intervals.
- **PoissonRegressor:** generalized linear model for nonnegative **counts/rates** with log link; counts, exposure and overdispersion need domain modeling.
- **GammaRegressor:** positive continuous skewed outcomes with suitable distributional assumptions.
- **TweedieRegressor:** family covering various distributions, including compound positive/zero contexts (power controls variance relation).

```
from sklearn.linear_model import HuberRegressor, PoissonRegressor, QuantileRegressor
# HuberRegressor for robust linear regression, scale numeric inputs first.
# PoissonRegressor for nonnegative count y, with appropriate exposure specification.
# QuantileRegressor(quantile=.9) estimates conditional upper 90% quantile.
```

Soutenance: "Why not MSE for every target?" Because target support and error consequences may differ. A count model understands nonnegative discrete outcome structure; a quantile model can target one-sided operational risk.

C. Time series forecasting without deep learning

Predicting the future requires time-aware features and evaluation: lag y_{t-1}, rolling mean of **past** values, calendar variables, weather known at issuance time, holiday flags. Classical approaches: **seasonal naïve** ($\hat{y}_t = y_{t-s}$), autoregression, ARIMA/SARIMA (`statsmodels`), exponential smoothing, and standard tabular ML (Ridge, forests, boosted trees on lags).

```
# Series indexed by time, sorted oldest → newest
# df['lag_1'] = df['sales'].shift(1)
# df['rolling_7_past'] = df['sales'].shift(1).rolling(7).mean()
# df['day_of_week'] = df.index.dayofweek
# Split chronologically, then fit full pipeline; avoid randomly shuffled CV.
```

Rolling origin evaluation: Train up to time t, evaluate on $t+1 \dots t+h$; advance origin, repeat. Forecast horizon, overlapping labels and latency define gaps; `TimeSeriesSplit` is a starting tool, not a guarantee against leakage. **Metric:** MAE/RMSE, sometimes MASE for comparing to seasonal naïve, with careful denominator and seasonal context. Weather forecast observations issued *after* the demand prediction time are forbidden as features.

D. Semi-supervised learning and active learning

Semi-supervised: use limited labeled plus many unlabeled observations under data distribution assumptions. `SelfTrainingClassifier` can pseudo-label high-confidence unlabeled samples, but confirmation bias from wrong pseudo-labels is a risk. `LabelPropagation` and `LabelSpreading` use a graph of sample similarities; expensive and distance-sensitive. **Active learning:** select uncertain/informative samples for a human oracle to label, balancing representation, costs and potential biases. Neither replaces careful independent labeled evaluation.

```
from sklearn.semi_supervised import SelfTrainingClassifier
from sklearn.linear_model import LogisticRegression
semi = SelfTrainingClassifier(LogisticRegression(max_iter=1000), threshold=.85)
# Requires X with correct numeric preprocessing and y containing -1 for unlabeled rows.
# semi.fit(X_train, y_partially_labeled)
```

E. Association rules and frequent patterns

Apriori / FP-Growth: Find frequent itemsets in basket transactions. Rule $A \Rightarrow B$: **support** $P(A \cap B)$, **confidence** $P(B \mid A)$, **lift** $P(A \cap B)/(P(A)P(B))$. Lift above 1 indicates positive association relative to independence under that dataset; it does not show causal influence. Association mining is distinct from supervised classification.

Example: 100 baskets, 30 contain bread, 20 contain butter, 12 contain both. Support=12%, confidence bread→butter=12/30=40%, lift=.12/(.30·.20)=2.0. Interpret segment/population composition carefully.

F. Recommendation and ranking

Content-based filtering: use feature similarity between an item and liked items. **Collaborative filtering:** use user-item interaction matrix; **matrix factorization/SVD** learns low-dimensional user/item factors (not always standard `TruncatedSVD` for missing ratings). **Learning-to-rank:** optimize ordering of candidates, not a single label; common metrics include `Precision@k`, `Recall@k`, `MAP`, `MRR` and `NDCG@k`.

Ranking challenge: Sorting by predicted purchase probability may not optimize diversity, novelty or user satisfaction; evaluation requires time-aware holdouts and exposure-bias consideration. In ranked datasets, grouping by query/user context is essential; do not scatter the same query across train/test without design.

G. Survival analysis and censoring

In "time until churn/failure", many customers **have not yet churned** when data collection ends. Treating all currently active customers as never-churn may bias a normal binary classification. Survival analysis models the event hazard/survival curve and accounts for right censoring; classics include Kaplan–Meier estimation and Cox proportional hazards. Hazard ratio interpretation requires assumptions such as proportional hazards. `lifelines` is a common Python option (ex-

tra optional dependency). Consider survival modeling when the *timing of an event* and censoring matter.

H. Multioutput, sample weights, cost and distribution shift

`MultiOutputRegressor` fits separate regressors per target; `MultiOutputClassifier` for multiple outputs; `OneVsRestClassifier` for multilabel classification. Not all estimators support `sample_weight` or output probabilities in the same way. Weighting rare classes during fit optimizes a *different objective*, may change calibration and is not the same as moving a decision threshold after training.

Distribution shift: $P_{train}(X, Y)$ differs from $P_{deploy}(X, Y)$. Distinguish covariate, label and concept shifts and ask which changes can be detected without ground truth. Reweighting data assumes certain relationships remain stable; it cannot solve arbitrary concept drift.

I. Predictive uncertainty and conformal methods

Point predictions alone hide uncertainty. Distinguish uncertainty in estimated model parameters vs inherent noise; regression residual analysis, bootstrap predictions, model ensembles, quantile regression and conformal prediction can produce intervals. **Conformal prediction** often uses held-out calibration residuals to obtain marginal coverage under exchangeability assumptions; it does **not** guarantee conditional coverage for every subgroup/individual. Calibration and distribution shift must be discussed.

Soutenance takeaway: More algorithm names do not make a model professional. Know what quantity the method estimates, what assumptions it needs, what dataset split makes evaluation valid, and what its score means for the decision.

Official References and API Verification

These **official documentation links** support the library patterns in the handbook. Machine-learning APIs evolve; inspect the version installed on your machine before running older code.

Core Python and scikit-learn

- NumPy reference
- Pandas user guide
- SciPy documentation
- scikit-learn User Guide
- Common pitfalls / data leakage
- Pipelines / ColumnTransformer
- Cross-validation
- Metrics API
- Supervised learning
- Unsupervised learning
- Clustering
- Probability calibration
- Model persistence
- imbalanced-learn

Gradient boosting, interpretation and visualization

- XGBoost Python API
- LightGBM LGBMRegressor reference
- LightGBM LGBMClassifier reference
- CatBoost Python fit reference
- UMAP documentation
- SHAP documentation

Tracking and deployment

- MLflow Tracking quickstart
- MLflow Tracking API
- FastAPI tutorial
- Streamlit docs

- Joblib persistence

Other classical methods

- Statsmodels
- lifelines survival analysis
- scikit-learn Semi-supervised learning
- scikit-learn Inspection

Version-conscious notes (November 2026):

1. Newer scikit-learn provides `root_mean_squared_error`; for older releases, `np.sqrt(mean_squared_err` is a compatible equivalent.
2. Some examples on the internet use outdated `OneHotEncoder(sparse=...)`; current scikit-learn versions use `sparse_output=` when that parameter is needed. This book uses defaults unless necessary.
3. MLflow documentation uses `mlflow.sklearn.log_model(sk_model=..., name=...)` in recent examples. Adapt to your installed version.
4. Scikit-learn `joblib`/pickle loading is not safe for untrusted artifacts and is not guaranteed cross-version compatible.
5. XGBoost, LightGBM, CatBoost, SHAP and UMAP package-specific data type/early-stopping interfaces may change independently; inspect their docs before adapting recipes.
6. `np.std` vs pandas `.std` differ in their default `ddof`; mathematical examples explicitly say population or sample std.

Documentation is a reference, not a substitute for empirical validation. The code files in this package are original educational examples.

Case Study 1 — Price Prediction (Regression)

Task: Predict a numeric selling price from vehicle (or property) attributes. **Do not treat a high R^2 as proof of a profitable pricing decision.** Define currency, inflation/date range and when attributes are known.

1. Business framing and target

- Observation unit: one listing or one completed sale, not an individual duplicated listing snapshot without safeguards.
- Target: final transaction price, if available; listing asking price answers a different question.
- Prediction date: when the model is used to estimate price.
- Main loss: MAE in currency, plus RMSE to surface large misses. Segment diagnostics by cheap/expensive and old/new vehicles.
- Baseline: median or mean of training prices (`DummyRegressor`).

2. Data dictionary example

Feature	Type	Transformation / concerns
`year`	numeric	Age at listing date; beware future records
`km_driven`	numeric	Skew; possible log transform and invalid negatives
`fuel`	nominal category	One-hot; unseen category at inference
`transmission`	nominal category	One-hot
`seller_type`	nominal category	One-hot
`owner`	ordinal or nominal	Confirm meaning before encoding
`brand`	nominal category	Derived from model text; high-cardinality issues
`selling_price`	target	Numeric currency, outliers, zeros and price definition

3. EDA plan

Inspect sample count, date range, missingness, impossible negative mileage, duplicate listings, target histogram/quantiles, scatter `km_driven` vs `selling_price`, target vs vehicle age, price by fuel/transmission, and rare brands. Plot age-price relationship separately by relevant groups; one global correlation can hide confounding. A high `year` coefficient may reflect model composition; it is not a causal guarantee.

4. Validation design

Random train/test on independent sale records is an initial demo. For realistic forward-deployment estimate, split by **sale/listing date** (train older, validate newer) and prevent near-duplicate vehicles from crossing partitions. Do not fit year-derived statistics/median imputers on full data. If you tune prices using the final test period, you need a *new* untouched future period to audit the locked method.

5. Modeling and scorecard

Compare `DummyRegressor`, `Ridge` (+ one-hot + StandardScaler numeric), `RandomForestRegressor` and `HistGradientBoostingRegressor` (with appropriate encoding). Use time-aware CV if data are temporal. Tune Ridge alpha, forest leaf size, boosting iterations/depth. Print CV mean and spread, then holdout **MAE, RMSE, R² and residual plots**. Do not compare scores that come from different split definitions.

```
# From the extracted handbook root:
# python code/regression_pipeline.py
# This runs an end-to-end synthetic housing-price version.
```

Why MAE vs RMSE? If real targets are `[100,200,300]` and predictions `[110,210,450]`, MAE is $170/3 \approx 56.7$, while RMSE is $\sqrt{(100 + 100 + 22500)/3} \approx 87.0$, reflecting the large final error more strongly. (Values illustrative.)

6. Oral defense questions

1. Why use `sell_price` instead of `advertised_price` as label? Answer hinges on intended real decision and availability.
2. Why could random split exaggerate generalization across years? Price regime shifts and duplicated listings.
3. Why not scale Random Forest? Threshold-based splits are largely order-invariant.
4. Why can't a forest extrapolate reliably far above the highest observed prices? Leaf predictions are averages of training targets.
5. What is a wrong model likely to do? Underestimate rare luxury vehicles, miss seasonal swings, fail on unseen brands.

Deliverable: A table of folds/results, two EDA plots, residual-by-price segment and a traceable final model artifact with model card.

Case Study 2 — Customer Churn Prediction (Classification)

Task: Predict whether an *active* customer will churn within the following 30 days from data available at a fixed prediction snapshot. This is not simply predicting whether an account *has ever closed*.

1. Important business questions

- What counts as churn? Cancellation, inactivity or billing cessation? Define consistently.
- When are we predicting? For example, each Monday at 08:00 for currently active customers.
- Label horizon: event within the next 30 days. Avoid right-censored recent periods without fully observed labels.
- What action follows a positive alert? Outreach or support; how many alerts can be reviewed?
- Are false alarms or missed churners more costly? Define cost or service-capacity constraint *before threshold tuning*.

2. Dataset

Field	Type	Risk
`tenure`	numeric	Calculation date must be snapshot date
`monthly_charge`	numeric	Currency, missingness, segment interactions
`contract_type`	nominal	Encoded with one-hot / native categories
`payment_method`	nominal	Missingness/unseen categories
`support_calls_last_30d`	count	Feature window must end before prediction
`satisfaction_score`	ordinal/numeric	Missingness and whether collected after churn
`cancelled_on`	date	**Leakage** if used as feature
`churn_next_30d`	binary target	Positive prevalence, label maturity

3. EDA and leakage audit

Inspect class rate, churn by contract, support call counts, tenure and target cross-tabulation, trends across monthly cohorts, missingness, duplicated customers, and class prevalence over time. A customer's post-cancellation support calls or "account_closed" field reveals outcomes; remove.

4. Split and pipeline

Choose **customer-grouped and/or chronological** validation if repeated snapshots exist; `StratifiedKFold` is only appropriate if rows are independent and exchangeable. Use numeric median+scaling for linear/KNN/SVM; categorical frequent imputation+OHE; keep inside `ColumnTransformer`. If oversampling, use a compatible fold-local imbalanced-learn pipeline, or test `class_weight` as simpler baseline.

5. Compare candidates

- `DummyClassifier(prior)` — ranks equal/prior probabilities and establishes prevalence AP.
- `LogisticRegression` — simple interpretable log-odds baseline.
- `RandomForestClassifier` — nonlinear interactions without scaling requirement.
- `HistGradientBoostingClassifier` / XGBoost / CatBoost — boosted tabular alternatives.

Primary ranking metric: AP if true positives are rare and alerts are important; **secondary:** ROC-AUC, positive recall, precision/F1 at chosen threshold, Brier score/calibration. Report deployment threshold separately from ranking metrics.

```
# End-to-end toy version with OOF threshold selection:
# python code/classification_pipeline.py
```

6. Threshold calculation example

On a **development validation set** with 100 churners and 900 non-churners:

Threshold	TP	FP	FN	TN	Precision	Recall
0.30	80	50	20	850	61.5%	80%
0.70	45	10	55	890	81.8%	45%

If cost is $10 \cdot FN + 1 \cdot FP$, threshold .30 costs 250 vs threshold .70 costs 560. This is an **illustrative** utility calculation, not an automatic business recommendation. A review-capacity constraint could favor fewer alerts regardless of raw cost.

7. Explain the result

Report **prevalence**, cross-validated AP with spread, final holdout AP/ROC, confusion matrix at fixed threshold, calibration and subgroup breakdown. Do not claim that an AUC of 0.9 means 90%

of customer decisions are correct. Distinguish correlation/association from causality: targeting high-risk customers does not necessarily cause retention.

Oral challenges: Why choose AP instead of accuracy? How did you select threshold without looking at test? What changes if customers appear every month? What if labels take 30 days to mature? How will you detect performance drop after a pricing-policy change?

Case Study 3 — Customer Segmentation (Unsupervised)

Task: Discover interpretable behavior groups, not predict predefined marketing labels. Cluster IDs are arbitrary indices and should be given meaningful descriptive names only *after* reviewing characteristic distributions.

1. Define similarity

Use customer-level summaries such as 90-day spending, purchase frequency, average basket size, time since last purchase and support-contact frequency. Do not include arbitrary `customer_id`. Normalize units and consider log transformations if spend has a long tail. Very correlated or redundant features may dominate Euclidean distance.

2. EDA and preparation

- Check missingness, duplicate customers, impossible negative transactions.
- Look at distribution per field and skew; decide meaningful observation window.
- Consider robust scaling for extreme monetary outliers; inspect whether outliers reflect legitimate VIP customers.
- Avoid mixing categorical numeric codes with numeric distances without design justification.

3. Compare K-Means, DBSCAN and GMM

K-Means: start $k = 2 \dots 8$, inspect inertia elbow, silhouette, cluster sizes and seed stability. Centroids provide representative profiles in standardized units; inverse transform for business presentation.

DBSCAN: tune `eps`, `min_samples`, inspect ratio of noise points and density-varying failure. Great for unusual, nonconvex shapes; standard sklearn DBSCAN lacks simple `predict` for new customers.

GMM: select component count/covariance via CV or AIC/BIC with care, examine posterior membership probabilities and class overlap. AIC/BIC prefer likelihood-parsimony under Gaussian assumptions, not a business segmentation truth.

```
# python code/clustering_demo.py
# Demonstrates inertia, silhouette, GMM BIC, DBSCAN noise and PCA.
```

4. Validate usefulness

Check whether clusters remain after small changes in training sample, seed, time window and feature scales. Evaluate profile sizes: a mathematically distinct segment of 2 people may not be actionable. Label segments as “frequent low-basket”, “infrequent high-basket” only if original-unit summaries support the labels. Do not call them predictive “churners” based only on unsupervised patterns.

5. Using clusters as supervised features

If segment ID is added to a churn classifier, fit the clusterer on each *training fold* within a reproducible process. Clustering on train+validation+test before CV is a leakage risk even without using labels. One solution is a train-fitted KMeans transformer that yields distances to centroids rather than arbitrary cluster code; distance features are continuous and have consistent centroid order for that particular fitted model.

```
from sklearn.cluster import KMeans
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
cluster_features = Pipeline([
  ('scale', StandardScaler()),
  ('centroid_distances', KMeans(n_clusters=4, n_init=10, random_state=42)),
```

```
  ('classifier', LogisticRegression(max_iter=1000)),
])
# KMeans.transform(X) returns distances-to-centroids; Pipeline uses transform before clf.
# cluster_features.fit(X_train_numeric, y_train)
```

6. Oral questions

Why is elbow ambiguous? Why does silhouette not establish ground truth? What distinguishes PCA from clustering? Can high-dimensional K-Means fail even after scaling? How would you assign clusters to new users? Does a new K-Means fit preserve cluster label indices? (No: IDs can permute.)

Deliverable: comparison of algorithms, selected representation, cluster profiling with original units, one PCA visualization with caveats, stability assessment and operational segmentation rationale.

Case Study 4 — Anomaly Detection + Operational Monitoring

Task: Flag observations unusual relative to historical behavior, with human review before consequential decisions. Anomaly ≠ mistake ≠ harmful activity. Ground the problem in actual reviewed outcomes when available.

1. Define abnormality and decision cost

Examples: equipment measurements outside typical operating patterns, unusual transaction timing, or exceptional logistics delay. Decide what is normal data distribution and whether available examples contain known anomalies. If normal-only training is plausible, novelty detection may fit; if normal data are contaminated, outlier detectors with robust assumptions are needed.

2. Data pipeline

1. Validate column units and schema, timestamps, duplicates, missingness.
2. Split chronologically to represent incoming observations; ensure events/related sensors don't straddle folds.
3. Fit train-only imputer/scaler where the detector requires it.
4. Fit IsolationForest, LOF or OneClassSVM using the appropriate normal/unlabeled training subset.
5. Select operational threshold using development review labels or budget constraints, **not** the test set.
6. Evaluate precision, recall, alert volume and false-positive patterns if ground truth exists.

3. Model comparison

Model	When useful	What can fail
Isolation Forest	Many numeric rows, global unusual patterns	Random isolation may flag rare but benign groups
LOF	Local density anomalies	Density differences, high-dimensional distance, standard fit-predict constraints
One-Class SVM	Smaller scaled datasets, nonlinear region	Sensitive to scaling, kernel and contamination assumptions

Code: `python code/clustering_demo.py` also demonstrates isolation-forest anomaly flags on generated data. An output of `-1` is a detector label, not verified truth.

4. Deployment interface

Load only a trusted versioned full pipeline, validate input schema with Pydantic, log model version and latency, avoid storing raw sensitive input fields in debug logs, and provide a review

pathway. `FastAPI` is appropriate for a request–prediction endpoint; `Streamlit` is appropriate for inspecting aggregate metrics and sample reports. The included examples show interface mechanics, not operational decisions.

5. Monitoring plan

- **Data quality:** null rate, ranges, categories, timestamp freshness, schema errors.
- **Output:** alert volume, score distribution, cases per subgroup or device type.
- **Delayed outcomes:** true alert performance among completed reviews, not among unresolved cases.
- **Operations:** p50/p95 prediction latency, HTTP errors, CPU/memory, model version.
- **Drift:** compare representative development/reference distributions with recent data; verify change is meaningful before retraining.

Oral questions: Does contamination set the true anomaly prevalence? (No.) What is novelty vs outlier detection? Why can you not trust anomaly clusters without feedback? How do you avoid turning statistical rarity into unfair treatment? What labels would you collect to validate later?
